

SLIMS API manual - 7.2

Generated on 30 October 2025

SLIMS API manual

by Agilent Technologies, Inc.

<pd1-slims-pse@agilent.com>

Generated on 30 October 2025

Table of Contents

Preface	ix
1. Introduction	1
1.1.	1
2. SLIMS Architecture	3
2.1. Architecture	3
2.2. Simplified database diagrams	3
2.2.1. Diagram for Standard licenses	3
2.2.2. Diagrams for Premium licenses	4
3. SLIMS GATE Development	6
3.1. Introduction to SLIMS GATE Plugins	6
3.2. Development Environment	6
3.2.1. Set up a Development Environment on your Local Machine	6
3.2.2. Files structure	7
3.2.2.1. Resources files	7
3.2.2.2. Source Files	7
3.2.2.3. build.gradle	8
3.2.2.4. Jasper Templates	8
3.2.3. Troubleshooting and Debugging	8
3.2.3.1. General Troubleshooting	8
3.2.3.2. Using a Remote Debugger	8
3.3. Start Example	11
3.3.1. Start Example <code>slimsgate.xml</code>	11
3.3.2. Start Example <code>camel-context.xml</code>	12
3.3.3. Start Example <code>plugin.properties</code>	12
3.3.4. <code>SpringConfig.java</code>	12
3.3.5. <code>CustomConfiguration.java</code>	13
3.3.6. <code>TestnameRouteBuilder.java</code>	13
3.3.7. Start Example Bean	14
3.4. Routes	15
3.4.1. Routes Associated with Flows Started from the GUI	15
3.4.2. Routes Associated with Automatically Started Flows	16
3.4.3. Useful Apache Components	16
3.5. Definition of <code>slimsgate.xml</code>	17
3.5.1. <i>Preamble</i> Elements	17
3.5.2. Definition of Steps	18
3.5.3. Input Parameters	19
3.5.4. Process	21
3.5.5. Output Parameters	21
3.5.6. Flow with Records Selection	22
3.5.7. Parameters of Type <code>SingleChoice</code> and <code>MultipleChoice</code>	23
3.5.8. Parameters of Type <code>SingleChoiceGrid</code> and <code>MultipleChoiceGrid</code>	27
3.5.9. Parameters of Table Type	30
3.5.10. Parameters of Configuration Type	31
3.6. SLIMS API	32
3.6.1. SLIMS plugin API overview	32
3.6.2. Database Interaction Example	34
3.6.3. Database Interaction Objects	38
3.6.3.1. Methods Summary	39
3.6.4. Database Queries Objects	40

3.6.4.1. <i>TableRecordQueries</i>	40
3.6.4.2. <i>TableQueries</i>	40
3.6.4.3. <i>UserQueries</i>	40
3.6.4.4. <i>AttachmentStore</i>	40
3.6.5. SlimsCriterion to narrow down FetchRequests	41
3.6.6. Setting the ActiveUser	41
3.6.7. SlimsGateErrorException	42
3.6.8. Checking for Flow Interruption	42
3.6.9. Working with Progress Bar	42
3.6.10. Trigger Experiment Run Step Refresh via SLIMS GATE	42
3.6.11. Use NGS Samplesheet Generation API in Plugins	42
3.6.12. Create Notifications via SLIMS GATE	43
3.6.13. Using TokenApi	43
3.6.14. Updating Quantities	44
3.6.15. Enrolling content in a study	45
3.6.16. Short Examples	45
3.6.16.1. Linking Content to Experiment Run Step	45
3.6.16.2. Create a Usage Instance	46
3.6.16.3. Generate a Jasper PDF Report Using Jasper API	47
3.6.16.4. Trigger Generation of Core Report in SLIMS GATE	47
3.7. SLIMS GATE flows and routes using the SLIMS REST API	48
3.7.1. Introduction	48
3.7.2. Exposing a SLIMS GATE route through SLIMS REST	49
3.7.2.1. Writing the plugin	49
3.7.2.2. Passing information to a plugin	50
3.7.2.3. Posting a body to the plugin	50
3.7.2.4. Running this asynchronously	51
3.7.3. Starting a SLIMS GATE flow via SLIMS REST	51
3.8. Using custom fonts in Jasper	52
3.8.1. Get a custom font jar file	52
3.8.2. Upload the jar file to the SLIMS server	52
3.8.3. Use the font in Jasper	52
4. SLIMS REST Development	53
4.1. Introduction	53
4.2. SLIMS REST API documentation	54
4.2.1. Automatic documentation	54
4.2.2. Offline documentation	54
4.2.3. Remarks	55
4.3. Examples of using REST Requests	55
5. SLIMS Python Development	57
5.1. SLIMS Python API	57
5.1.1. Python API installation	57
5.1.2. Python API upgrade	57
5.1.3. Python API versions	57
5.1.4. Python API Online Documentation	58
6. Groovy Script Development	59
6.1. Introduction to Groovy	59
6.2. Groovy Editor	60
6.2.1. Fields Where Groovy is Available	60
6.2.2. Editor	60
6.3. Introduction to Fluent Groovy API	61
6.3.1. Advantages of Fluent Groovy API	61

6.3.1.1. Convenience	61
6.3.1.2. Optimized by default	62
6.3.1.3. Human-readable syntax	62
6.3.2. How to Generally Build Fluent Groovy Expressions	64
6.4. Guidance on Fluent Groovy API	66
6.5. Build-in SLIMS Methods and Variables	67
6.5.1. oldValue	68
6.5.2. empty	68
6.5.3. NA	68
6.5.4. log	68
6.5.4.1. Methods Summary	68
6.5.4.2. log.error	69
6.5.4.3. log.warn	69
6.5.4.4. log.info	69
6.5.4.5. log.debug	69
6.5.5. tableName	70
6.5.5.1. Methods Summary	70
6.5.5.2. tableName.get	70
6.5.6. authenticatedUser	71
6.5.6.1. Methods Summary	71
6.5.6.2. authenticatedUser.groupNames	71
6.5.6.3. authenticatedUser.role_name	71
6.5.6.4. authenticatedUser.role_updateLevel	71
6.5.6.5. authenticatedUser.user_email	72
6.5.6.6. authenticatedUser.user_pk	72
6.5.6.7. authenticatedUser.user_userName	72
6.5.7. simultaneousChanges	72
6.5.8. Quantity and Unit	72
6.5.8.1. quantity(amount, unit)	72
6.5.8.2. unit(String abbreviation)	73
6.5.9. Comparator Quantity and Position	73
6.5.9.1. comparatorQuantity(position, amount, unit)	73
6.5.9.2. ComparatorQuantityPosition	74
6.5.10. triggerRecalculation	74
6.5.10.1. triggerRecalculation(entityName, recordPk) for fields	74
6.5.10.2. triggerRecalculation(entityName, recordPk, separateTransaction) for scheduled jobs	75
6.5.11. triggerDelayedRecalculation	76
6.5.12. triggerDelayedSlimsGateFlow	77
6.5.13. triggerDelayedEmail	78
6.5.14. findByUniqueIdentifier	78
6.5.15. macroExecutionDetails	78
6.5.16. daoHelper	79
6.5.16.1. Methods Summary	80
6.5.16.2. daoHelper.fetch	80
6.5.16.3. daoHelper.fetch	81
6.5.16.4. daoHelper.find	81
6.5.16.5. daoHelper.getFetchRequest	81
6.5.17. FetchRequest	82
6.5.17.1. Methods Summary	82
6.5.17.2. FetchRequest.setCriterion	82
6.5.17.3. FetchRequest.setAddSecurityCriterion	82

6.5.18. SlimsRestrictions	83
6.5.18.1. Methods Summary	83
6.5.18.2. SlimsRestrictions.equals	84
6.5.18.3. SlimsRestrictions gt/lt/ge/le	84
6.5.18.4. SlimsRestrictions.in	85
6.5.18.5. SlimsRestrictions.like	85
6.5.18.6. SlimsRestrictions.not	86
6.5.18.7. SlimsRestrictions.and	86
6.5.18.8. SlimsRestrictions.or	86
6.5.19. Fetch Contents Affecting/Affected By a Content	87
6.5.19.1. Fetch Content(s) Affected By a Content	87
6.5.19.2. Fetch Content(s) Affecting a Content	87
6.5.19.3. Fetch Both Content(s) Affecting and Affected By a Content	87
6.5.20. sequences	88
6.5.20.1. Methods Summary	88
6.5.20.2. sequences.next	88
6.5.21. ok	88
6.5.22. warn	89
6.5.23. error	89
6.5.24. notify	90
6.5.25. email	90
6.5.26. holidayHelper	91
6.6. Examples	91
6.6.1. Fields, datatype Quantity, table Content: calculate mass from Volume and Concentration field values	91
6.6.2. Fields, datatype Decimal number, table Content: calculate the difference between two quantity fields and converts it to double	92
6.6.3. Field <code>cntn_fk_group</code> , table Content: set default group if unique	92
6.6.4. Fields, datatype Date, table Content: calculate expiration date from pro- duction date and duration	92
6.6.5. Rules, Order Status: perform checks on requestables when scheduling an order	93
6.6.6. Rules, Conditional value expression: check that all results are verified when stopping a step	93
6.6.7. Rules, Conditional value expression: requires a value for a Protocol run step field when stopping the step	94
6.6.8. Rules, Conditional value expression: checks that all contents are re- moved when stopping a Protocol run step	94
6.6.9. Rules, Conditional value expression: use differencesMap to prevent field update based on Content status	94
6.6.10. Rules, Status: use differencesMap to prevent field update based on Or- der Status	95
6.6.11. Rules, Result Status: requires a comment for some kind of results	95
6.6.12. Fields, datatype Dynamic choice: example of Dynamic filter expression	95
6.6.13. Notify and Link to records: Generate links using GUID or UID	96
6.6.14. Test: calculate from values of other tests	96
6.6.15. Test result: calculate result for all results of quantity test	97
6.7. Test Groovy Scripts	97
6.8. Advanced groovy behaviour	98
6.8.1. Date comparisons	98

List of Figures

2.1. Diagram for Standard licenses	3
2.2. Diagram for Premium licenses overview	4
2.3. Diagram for Premium licenses for Workflows	5
2.4. Diagram for Premium licenses for Analytical Workflows	5
3.1. Create a new Remote configuration in IntelliJ	9
3.2. Change the name of the remote configuration in IntelliJ - Set the Host to the IP address of the SLIMS server and set the appropriate port	9
3.3. Create a new Remote configuration in Eclipse	10
3.4. Change the name of the remote configuration in Eclipse - Set the Host to the IP address of the SLIMS server and set the appropriate port	10
3.5. Flow pop-up window	11
3.6. If a flow requires a selection	23
3.7. Definition of <fieldList>	26
3.8. Definition of <valueField>	27
3.9. <type> Table Input	30
4.1. REST API documentation	54
6.1. Diagram with Generic Fluent Groovy API	64
6.2. Test value expression dependence	97

List of Tables

3.1.	24
3.2.	25
6.1. Permissions for Value Expressions	60
6.2. Build-in SLIMS Variables	67

Preface

This document is a reference guide for the SLIMS API to help Agilent and customer engineers programmatically interface with the SLIMS application.

1. Introduction

Manual Part Number

R4526A-DM7.2.8

30 October 2025

Copyright

2025 © Agilent Technologies, Inc.

No part of this manual may be reproduced in any form or by any means (including electronic storage and retrieval or translation into a foreign language) without prior agreement and written consent from Agilent Technologies, Inc. as governed by the United States and international copyright laws.

Agilent Technologies, Inc.

5301 Stevens Creek Blvd.

Santa Clara, CA 95051

USA

Software Revision

This guide is valid for the 7.2.8 revision of Agilent SLIMS software.

Warranty

The material contained in this document is provided 'as is,' and is subject to being changed, without notice, in future editions. Further, to the maximum extent permitted by applicable law, Agilent disclaims all warranties, either express or implied, with regard to this manual and any information contained herein, including but not limited to the implied warranties of merchantability and fitness for a particular purpose. Agilent shall not be liable for errors or for incidental or consequential damages in connection with the furnishing, use, or performance of this document or of any information contained herein. Should Agilent and the user have a separate written agreement with warranty terms covering the material in this document that conflict with these terms, the warranty terms in the separate agreement shall control.

Technology Licenses

The hardware and/or software described in this document are furnished under a license and may be used or copied only in accordance with the terms of such license.

Restricted Rights Legend

U.S. Government Restricted Rights. Software and technical data rights granted to the federal government include only those rights customarily provided to end user customers. Agilent provides this customary commercial license in Software and technical data pursuant to FAR 12.211 (Technical Data) and 12.212 (Computer Software) and, for the Department of Defense, DFARS 252.227-7105 (Technical Data - Commercial Items) and

DFARS 227.7202-3 (Rights in Commercial Computer Software or Computer Software Documentation).

Safety Notices

△ A **CAUTION** notice denotes a hazard. It calls attention to an operating procedure, practice, or the like that, if not correctly performed or adhered to, could result in damage to the product or loss of important data. Do not proceed beyond a **CAUTION** notice until the indicated conditions are fully understood and met.

△ A **WARNING** notice denotes a hazard. It calls attention to an operating procedure, practice, or the like that, if not correctly performed or adhered to, could result in personal injury or death. Do not proceed beyond a **WARNING** notice until the indicated conditions are fully understood and met.

2. SLIMS Architecture

2.1. Architecture

Details on server and software architecture are available in the Security and Architecture Policy for Agilent SLIMS 7.2

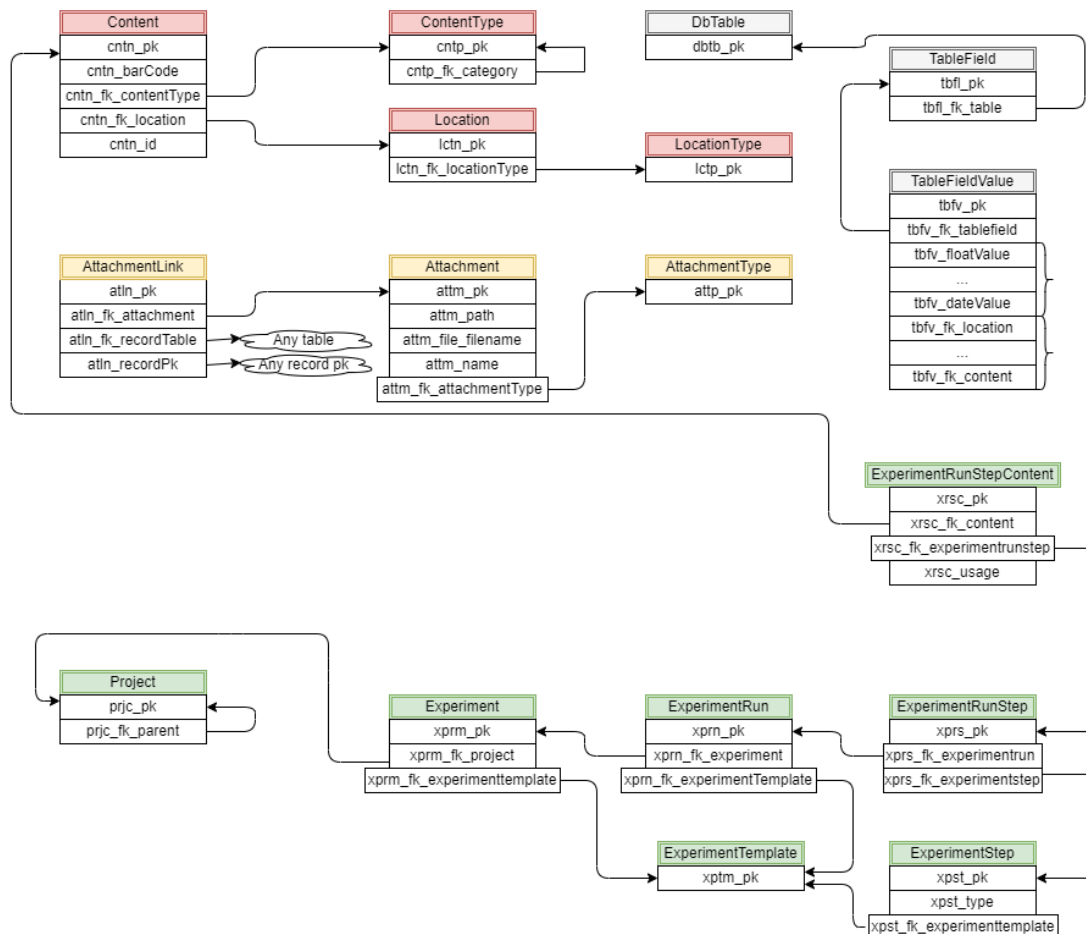
This document describes two APIs that are available for programmatically interacting with the SLIMS database.

- Chapter 3 describes how to use the SLIMS GATE plugins API and its parameters
- Chapter 4 describes how to use the SLIMS REST API and its parameters

2.2. Simplified database diagrams

2.2.1. Diagram for Standard licenses

Figure 2.1. Diagram for Standard licenses



2.2.2. Diagrams for Premium licenses

Figure 2.2. Diagram for Premium licenses overview

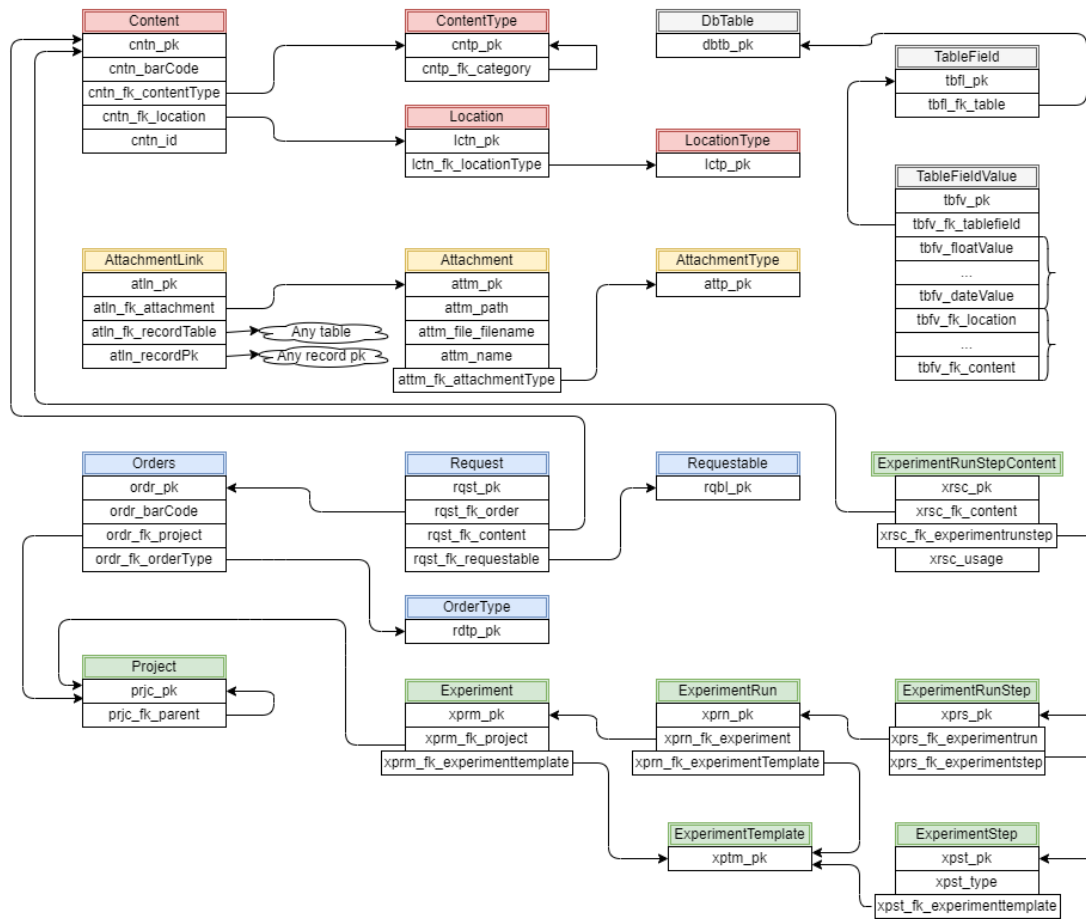


Figure 2.3. Diagram for Premium licenses for Workflows

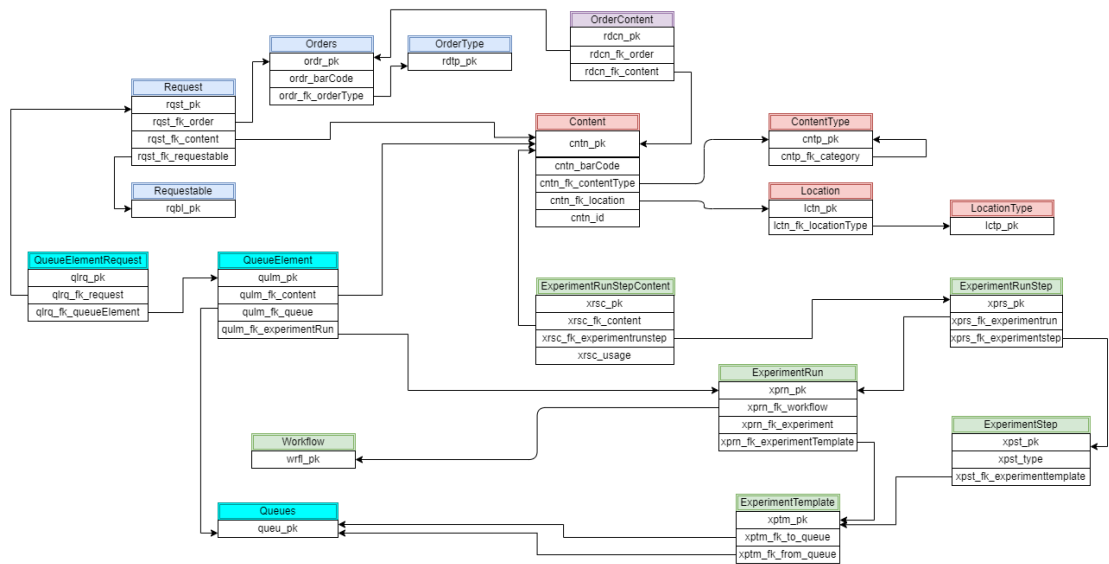
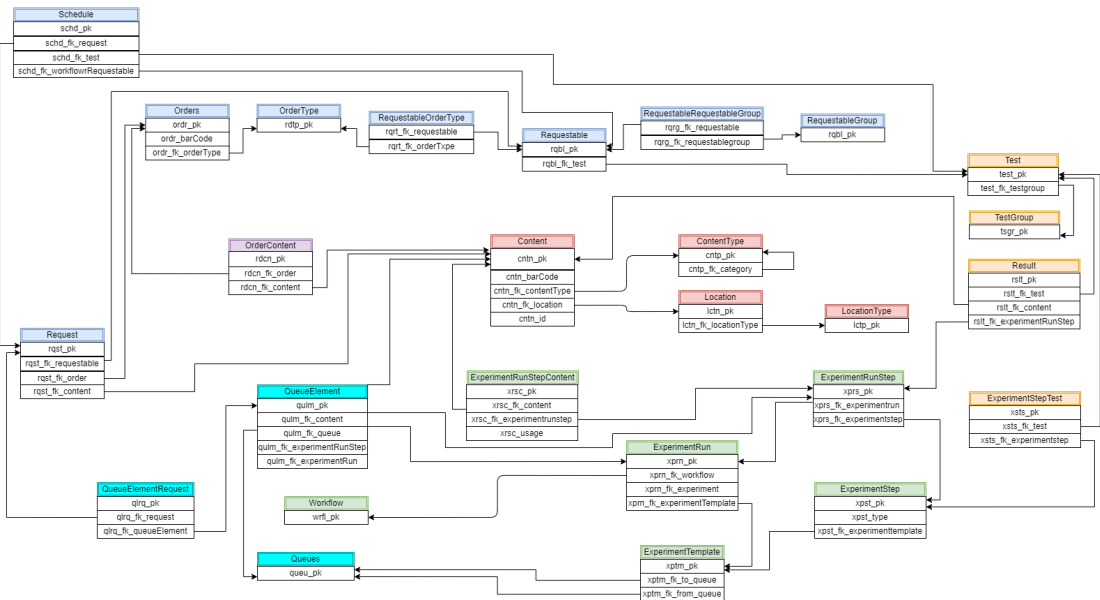


Figure 2.4. Diagram for Premium licenses for Analytical Workflows



3. SLIMS GATE Development

SLIMS GATE plugins development targets engineers who wish to create their own plugins. Using SLIMS GATE plugins it is possible to use a complete DAO API to interact with the SLIMS database.

3.1. Introduction to SLIMS GATE Plugins

The purpose of SLIMS GATE is to add functionalities to core SLIMS, such as to enable communication with the outside world (instruments, printers, mail, etc.) or to implement customer-specific business logic.

Each functionality will be added as a plugin. A base SLIMS GATE is delivered by default with SLIMS and can run the plugins (existing legacy SLIMS GATE from older SLIMS versions can also run the new plugins). A plugin is a piece of code responsible for one stand-alone functionality.

The plugins have been introduced in order to:

- Unlink SLIMS upgrade from SLIMS GATE upgrades
- Isolate code developed by the customer from SLIMS code
- Improve reusability of standard and custom functionalities

Whether they are delivered by Agilent or produced by the customer, plugins can be uploaded on a test instance through the SLIMS Plugins module. A plugin can have parameters that can be adjusted directly in the UI of the Plugins module. The plugins are ready to be used instantly after they are uploaded.

⚠ Agilent provides a **SLIMS Advanced Customization Training, Part number: R4536A** that must be completed before you can start creating plugins.

Plugins uploaded on a SLIMS instance are isolated from one another. Plugin A will not know that Plugin B exists, and vice versa. (It cannot access the code of plugin B).

Warning about SLIMS major upgrades:

After a major upgrade of SLIMS, changes to all customizations might be required.

These changes can be found in the install/upgrade manual under the customization development upgrade steps chapters

3.2. Development Environment

3.2.1. Set up a Development Environment on your Local Machine

Plugins can be developed and built on the local development machine and uploaded to SLIMS development server through SLIMS REST or SLIMS UI. Example plugins can be found on

<https://github.com/genohm/slims-api>. More detailed procedures are available during the SLIMS Advanced Customization Training (Part number: R4536A).

3.2.2. Files structure

Each plugin must have its own folder. In this folder, we must find:

- A `resources` folder
- A `src` folder for code
- A `build.gradle` file

3.2.2.1. Resources files

3 configuration files are found in this folder:

- The `slimsgate.xml` file in the `resources` folder is where flows are defined as a succession of steps. Each step has:
 - A set of inputs,
 - A process which refers to a route defined in the other configuration file (`camel-context.xml`),
 - And an output.

The syntax of this file is defined by Agilent, as explained in Section 3.5.

- The `camel-context.xml` file in the `resources/spring` folder connects the route name defined in the `slimsgate.xml` file with a routebuilder defined in the `SpringConfig.java` file. The syntax of this file is defined by Apache [<http://camel.apache.org>].
- The `plugin.properties` file in the `resources` folder is where the type of the plugin (SLIMS, SLIMS GATE, LIBRARY, or REMOTE) is set and the Java class defining the plugin parameters is referenced.

3.2.2.2. Source Files

Java source code developed by the customer is contained in the `src` folder.

A package named `com.genohm.slims.custom` should be created.

- The sub-package named `com.genohm.slims.custom.beans` will contain:
 - Your route builders: the syntax of this file is defined by Apache [<http://camel.apache.org>] and is explained in Section 3.4
 - Your custom beans: your actual code
- The file `SpringConfig.java` must be defined in `com.genohm.slims.custom` and is fixed.
- The file `CustomConfiguration.java` must be defined in `com.genohm.slims.custom` and defines the plugin parameters and default values. The name can be customized.

Java source code developed by Agilent is accessible via integrated tools in your chosen development environment (IDE), but is not editable.

3.2.2.3. build.gradle

The build.gradle file is empty. Dependencies are fetched automatically when building the project.

3.2.2.4. Jasper Templates

Jasper templates used by Reports of type SLIMS GATE can be committed under the folder `conf/META-INF/jasper/`. Each template (e.g. `report.jrxml`) needs to be put in a folder with the same name as the template without the extension (e.g. `report/`). This folder will contain also the related files and subreports (if any).

3.2.3. Troubleshooting and Debugging

3.2.3.1. General Troubleshooting

While developing, it is useful to know that you must re-build and re-upload the fatjar for any modification to be taken into account. In addition:

- If `slimsgate.xml` has been modified then, SLIMS should be refreshed.
- If the plugin configuration has been modified (with additional or removed parameters), then the plugin configuration should be updated.
- Errors are sent to the `slimsgate.log` file on the development server, the log tab in the plugins module of the SLIMS UI and can also be visualized by clicking on the top-right Bell icon of the SLIMS UI.

3.2.3.2. Using a Remote Debugger

This section describes how a Remote Debugger can be configured for the SlimsGate service so that plugin code can be debugged from the IDE.

1. Make sure that on the server running SlimsGate the port to use for the debugger is open to outside connections (default is 5005)
2. Have accessibility to the server running SlimsGate from the machine on which the debugger is run
3. Add a remote JVM Debug run configuration to your IDE with the IP of the server running SlimsGate and the chosen port. Assign this run configuration a name. If needed, multiple run configurations can be set up.
4. Add the command line arguments for remote JVM parameters to the SlimsGate java options (see also the Command line arguments for remote JVM in your IDE's run configuration). The default command line arguments from an IntelliJ debugger using the default port (5005) are:

```
-agentlib:jdwp=transport=dt_socket,server=y,suspend=n,address*=5005
```

changing the `suspend=n` to `suspend=y` makes the server startup stop and wait

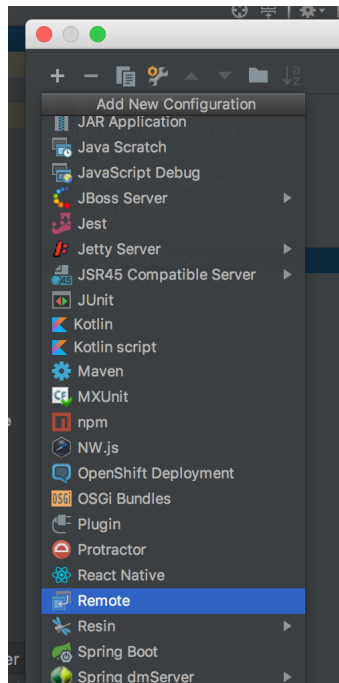
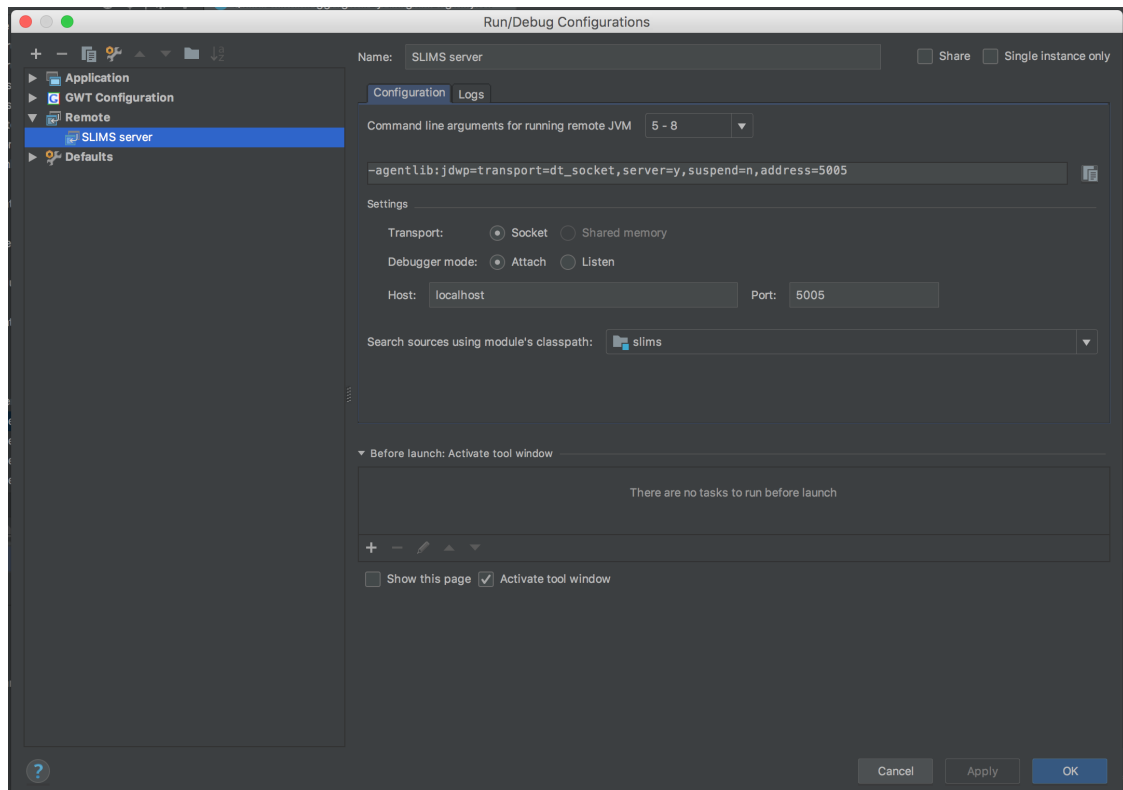
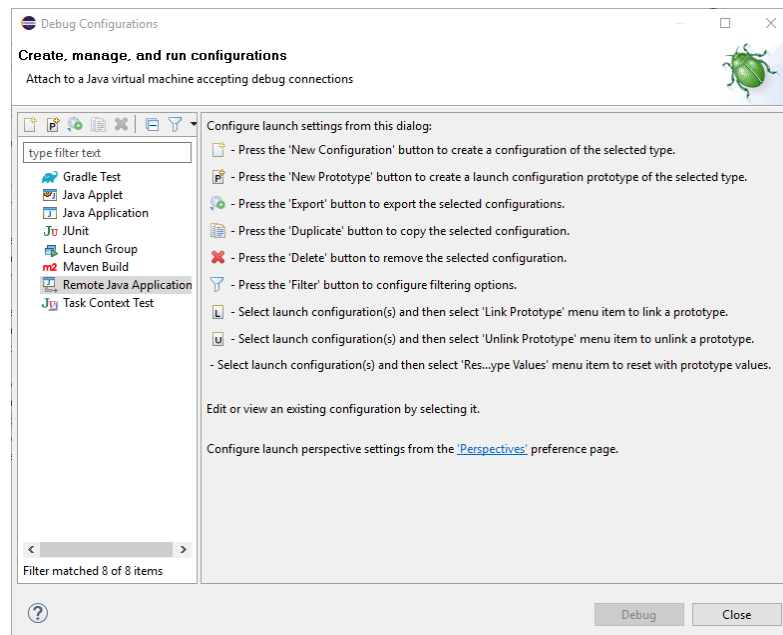
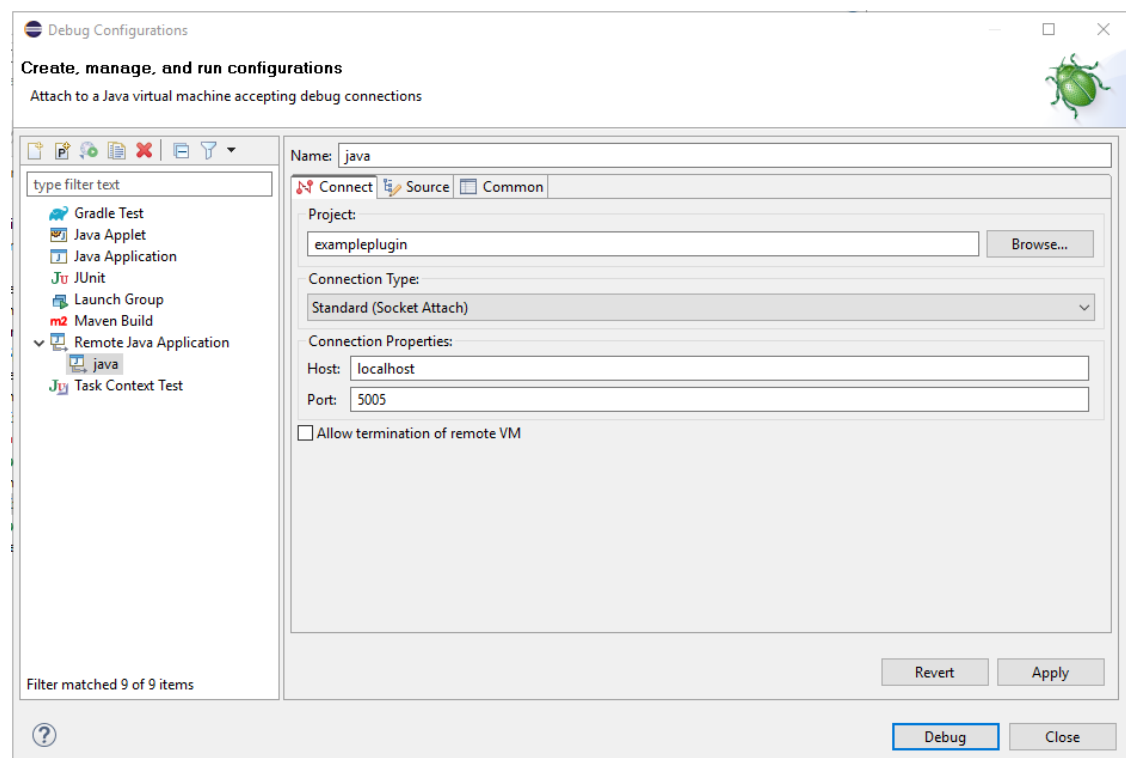
Figure 3.1. Create a new Remote configuration in IntelliJ**Figure 3.2. Change the name of the remote configuration in IntelliJ - Set the Host to the IP address of the SLIMS server and set the appropriate port**

Figure 3.3. Create a new Remote configuration in Eclipse**Figure 3.4. Change the name of the remote configuration in Eclipse - Set the Host to the IP address of the SLIMS server and set the appropriate port**

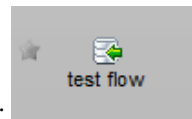
3.3. Start Example

You can download an example SLIMS GATE plugin from GitHub [<https://github.com/p00by/slims-api/tree/master/templates/slimsgate-template>]. This example illustrates:

- the complete structure of a flow, including the definition of a bean,
- how the `<input>` message can be read in the bean and
- elementary interaction with the database.

To run this example, fetch this example plugin from GitHub as follows.

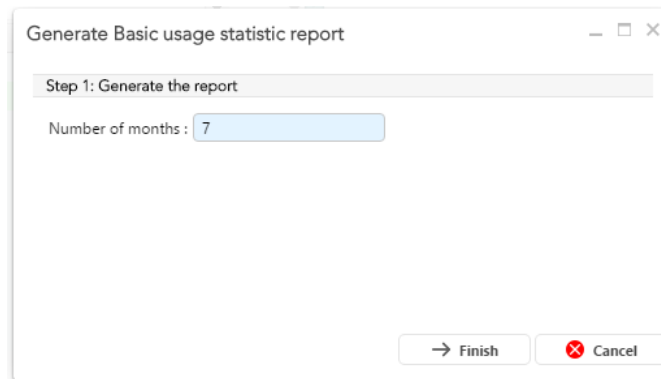
1. Copy the `<slimsgate-flow>` definition in `slimsgate-ext.xml`.
2. Copy the `<route>` definition in the `<camelContext>` block of `camel-context-ext.xml`.
3. Copy the `<bean>` call after the `<camelContext>` block of the same file.
4. Copy the definition of the `NameTestBean` in the appropriate file.



This flow can be called from the *Content Management* table:

The following *flow pop-up window* will then appear:

Figure 3.5. Flow pop-up window



3.3.1. Start Example `slimsgate.xml`

The **definition of the SLIMS GATE flow** to be reported in `slimsgate.xml` is

```
<slimsgate-flow>
  <id>slimsgate-test-flow</id>
  <name>test flow</name>
  <description>A test flow</description>
  <usage>contentmanagement</usage>
```

```

<step>
  <name>Write your name</name>
  <input>
    <parameter>
      <label>Your name</label>      <!-- Shown to the user -->
      <name>parameterName</name>    <!-- identifier to be used in the bean -->
      <description>What' s your name?</description> <!-- help text displayed when hovering over
the input -->
      <type>String</type> <!-- type of the input -->
    </parameter>
  </input>
  <process>
    <route>slimsgate-test-flow</route> <!-- The ID of the route to define in camel-context.xml
-->
  </process>
</step>
</slimsgate-flow>

```

3.3.2. Start Example camel-context.xml

The definition of `<route>` to be reported in `camel-context.xml` is:

```

<beans xmlns="http://www.springframework.org/schema/beans"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="
    http://www.springframework.org/schema/beans
    http://www.springframework.org/schema/beans/spring-beans-2.5.xsd
    http://camel.apache.org/schema/spring
    http://camel.apache.org/schema/spring/camel-spring.xsd">

  <camelContext id="camel-context" xmlns="http://camel.apache.org/schema/spring">
    <routeBuilder ref="testnameRouteBuilder"/>
  </camelContext>

</beans>

```

3.3.3. Start Example plugin.properties

The config-class must reference the class where the plugin parameters are defined.

```

config-class=com.genohm.slims.custom.CustomConfiguration
type=SLIMSGATE
apiVersion=7.2.0

```

3.3.4. SpringConfig.java

The content of this file is fixed.

```

package com.genohm.slims.custom;

import org.springframework.context.annotation.ComponentScan;
import org.springframework.context.annotation.Configuration;
import org.springframework.context.annotation.ImportResource;

@Configuration
@ComponentScan({"com.genohm.slims.custom", "com.genohm.slimsgate.service"})
@ImportResource("classpath:spring/camel-context.xml")
public class SpringConfig {

```

```
}
```

3.3.5. CustomConfiguration.java

In this class, you must declare 1 property for each variable that you want to be able to customize after your plugin is uploaded.

You must define a getter for each property.

```
package com.genohm.slims.custom;

public class CustomConfiguration {

    private Long subjectContentTypePk;
    private String myVariable;
    // Define all the parameters you need, it can be complex types

    public CustomConfiguration() {

    }

    public Long getSubjectContentTypePk() {
        return subjectContentTypePk;
    }

    public String getMyVariable() {
        return myVariable;
    }

    public static CustomConfiguration getDefault() {
        CustomConfiguration customConfiguration = new CustomConfiguration();
        customConfiguration.subjectContentTypePk = 10L;
        customConfiguration.myVariable = "OK";
        return customConfiguration;
    }
}
```

3.3.6. TestnameRouteBuilder.java

The name of this class has to be identical to the name of the route builder referenced in camel-context.xml In this class, you must define how to build your processing route. The syntax of this code is defined on Apache [<http://camel.apache.org>]

```
package com.genohm.slims.custom.beans;

import org.apache.camel.builder.RouteBuilder;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Component;

@Component
public class TestnameRouteBuilder extends RouteBuilder {

    @Autowired
    private NametestBean nametestBean;

    @Override
    public void configure() {
        createRoute();
    }
}
```

```
}

private void createRoute() {
    from("direct:slimsgate-testname") // the name after direct must be unique in this plugin
    (TBC?)
        .routeId("slimsgate-testname") // the name of the route defined in the slimsgate.xml
        .bean(nametestBean); // the bean that will process the inputs
}
}
```

3.3.7. Start Example Bean

The **@Component** annotation must be set on the class

```
package com.genohm.slimsgate.camel.beans.customer;
import java.util.List;
import java.util.Map;

import org.apache.camel.Handler;
import org.apache.camel.Header;
import org.slf4j.Logger;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.transaction.annotation.Transactional;

import com.genohm.slims.common.model.Content;
import com.genohm.slims.common.model.ContentMeta;
import com.genohm.slims.common.util.StringUtil;
import com.genohm.slims.server.dao.common.Dao;
import com.genohm.slims.server.repository.queriers.ContentRecordQueries;
import com.genohm.slimsgate.camel.gatekeeper.SlimsGateErrorException;
import com.genohm.slimsgate.camel.gatekeeper.SlimsGateKeeper;
import com.genohm.slimsgate.camel.gatekeeper.SlimsProxy;
import com.genohm.slimsgateclient.workflow.SlimsFlowInitParam;
import com.google.common.base.Optional;

@Component
public class NametestBean {

    @Autowired // Injects the instance of the following type. Always
    use with Dao and RecordQueries types.
    private Dao<Content> contentDao ; // use Dao<Tablename> for ADD, UPDATE and REMOVE
    records
    @Autowired
    private ContentRecordQueries contentRecordQueries ; // use TablenameRecordQueries for
    FETCH
    @Autowired
    private CustomConfiguration customConfiguration; // the configuration class

    @Transactional(rollbackFor = Exception.class) // Access the database and rollback
    in case an exception occurs
    @Handler // Defines which method is called by the
    route
    public void run(
        @Header(SlimsGateKeeper.SLIMS_WORKFLOW_INIT_PARAMETER) SlimsFlowInitParam initParam, //
        object containing input parameters, like inputs, user...
        @Header(SlimsGateKeeper.SLIMS_PROXY) SlimsProxy slimsproxy //
        for logging
    )
    {
        Logger logger = slimsproxy.getLogger(getClass()); // Gets the
        logger
        Map<String,Object> inputParameters = initParam.getInputParameterValues(); // Gets the
        input parameters
    }
}
```

```

String yourName = StringUtil.getAsString(inputParameters.get("parameterName")); //
Retrieves the input parameter "parameterName" defined in slimsgate.xml

Specify the package for com.genohm.slims.common.util.StringUtil

System.err.println(inputParameters); // Prints all input parameters in the log file
(useful when debugging)
logger.info(inputParameters.toString()); // Prints input parameters in the log GUI AND the
log file

System.err.println(inputParameters); // Prints all input parameters in the log file
(useful when debugging)
logger.info(inputParameters); // Prints input parameters in the log GUI AND the
log file

Optional<Map<String, Object>> contentByBarcode =
contentRecordQueries.findByBarCode(yourName); // when one or zero, returns an
com.google.common.base.Optional
List<Map<String, Object>> contentById = contentRecordQueries.findById(yourName);
// when many or zero returns a list (which maybe empty)

// recordQueries never return null!
if (contentByBarcode.isPresent()) {
    Map<String, Object> actualContent = contentByBarcode.get(); // records are
always retrieved as Map<String, Object>
    // modify the map as you need to
    actualContent.put("cntn_cf_test1", "TEST1");
    actualContent.put("cntn_cf_test2", customConfiguration.getMyVariable()); // get
customizable values from customConfiguration object
    // modify the map as you need to
    contentDao.update(actualContent ); // update the
content
} else {
    throw new SlimsGateErrorException("Hey, that does not exists!"); // Shows error to
user
}
}
}

```

3.4. Routes

Routes are part of a big topic, described in details at Apache Camel [<http://camel.apache.org>]. Here we only define general rules used in the definition of most of our routes.

3.4.1. Routes Associated with Flows Started from the GUI

When a route is initiated by a flow, the source (`from()` attribute) has to be set to "direct:someUniqueName" where someUniqueName can not be used in more than one route definition. Most of the time, the destination (`bean()` attribute) is set to a bean that is auto-wired in this route builder class.

It is possible to chain multiple beans, and to specify the process method. For example:

```

private void createRoute() {
    from("direct:slimsgate-testname") // the name after direct must be unique in this plugin
(TBC?)
        .routeId("slimsgate-testname") // the name of the route defined in the slimsgate.xml
        .bean(process1Bean)
        .bean(process2Bean, "methodName");
}

```


3.4.2. Routes Associated with Automatically Started Flows

It is possible to define routes automatically related to a certain event, using Camel Components [<http://camel.apache.org>]. In this case, the `from` attribute in the route definition will be other than a `direct` component, as it is for manually started routes. Also no GUI component needs to be defined in the `slimsgate.xml` file.

Some common examples:

- Start a route when a file is dropped in a defined folder. Example: the following route makes the flow to start when a file is added to `/folder/watched/by/SLIMSGate/`

```
private void createRoute() {
    from("file:/folder/watched/by/SLIMSGate/")
    .routeId("slimsgate-processFile")
    .bean(processFileBean);
}
```

Refer to <http://camel.apache.org/file.html>.

- Start a route at regular times. Example: the following route makes the flow to start every day at 11 pm

```
private void createRoute() {
    from("quartz://timerName?cron=0+0/5+12-18+?+*+MON-FRI")
    .routeId("slimsgate-cronProcess")
    .bean(processFileBean);
}
```

Refer to <http://camel.apache.org/quartz.html>.

3.4.3. Useful Apache Components

Several apache components exist and can be used as part of a route definition. These are some common use cases. (We recommend the full apache documentation for reference online at: <http://camel.apache.org>.)

- Watching a folder and reading files as soon as they are dropped in, as mentioned in the previous paragraph. Refer to <http://camel.apache.org/file.html>.
- Example of writing files:

```
private void createRoute() {
    from("direct:slimsgate-xxx")
    .routeId("slimsgate-xxx")
    .bean(producingFileBean)
    .to("file:/destination/folder/?fileName=nameOfFileIncludingTheDate_
    ${date:now:yyyyMMdd}.xml");
}
```

Refer to <http://camel.apache.org/file.html>.

- Example of sending mail:

```
private void createRoute() {
    from("direct:slimsgate-xxx")
```

```

        .routeId("slimsgate-xxx")
        .bean(producingMailBean)
        .to("smtp://a.mail.server.ip?contentType=text/html");
    }

```

Refer to <http://camel.apache.org/mail.html>

- Example of access remote file systems:

```

private void createRoute() {
    from("direct:slimsgate-xxx")
        .routeId("slimsgate-xxx")
        .bean(producingFileBean)
        .to("sftp://remote.server.name/folderName");
}

```

Refer to <http://camel.apache.org/ftp.html>

- Converting from or to some data format, refer to <http://camel.apache.org/data-format.html>.
Example of reading a CSV file:

```

CsvDataFormat csv = new CsvDataFormat();
csv.setDelimiter("|");

private void createRoute() {
    from("file:/folder/watched/by/SLIMSGate/")
        .routeId("slimsgate-xxx")
        .bean(processBean)
        .unmarshal(csv);
}

```

In this case the CSV file is transformed in a list of Strings. Refer to <http://camel.apache.org/csv.html>

3.5. Definition of slimsgate.xml

The definition of a SLIMS GATE flow is contained in one block `<slimsgate-flow>` and contains some required elements as a preamble and a series of one or more `<step>`s:

```

<slimsgate-flow>
  preamble elements: <id>, <name>, <description>, <usage>
  <step> ... </step>
  <step> ... </step>
  ...
</slimsgate-flow>

```

3.5.1. Preamble Elements

The **preamble** contains following elements:

- **<id>**: *required*. A unique identifier of the flow.
- **<name>**: *required*. The name appearing in the *start button*.
- **<description>**: *required*. It is used as a short help.
- **<usage>**: *required*. It indicates in which table the flow can be called. Possible values are: contentmanagement, eln, elntree, worklisttemplate, attachment, ordermanagement, resultmeasurement, internal, simplified, workflow, dashboard, macro and stabilitystudy.

- **<active>**: *optional*. If set to false, the flow is not active.
- **<icon>**: *optional*. Changes the icon of the SLIMS GATE flow, direct link to SLIMS/war/images folder.
- **<showProgress>**: *optional*. If set to true, a progress bar is available on top of the flow execution window, if the progress is defined in the bean. See Section 3.6.9 for more details.
- **<restriction>**: *optional*. It restricts the flow to specific roles, for example:

```
<restriction type="role">
  <allow>administrator</allow>
</restriction>
```

- `type` is a mandatory attribute. For now, it has only one possible value ("role").
- Leaving `<restriction>` empty will block all roles. However, flows without restriction tag will not be blocked at all.
- `<allow>` is the only possible child tag for a role restriction. Anyone not allowed is blocked.
- **<interruptable>**: *optional*. If set to true, the flow can be interrupted (see Section 3.6.8).
- **<repeatable>**: *optional*. If set to true, a "Finish and restart" button is available on the last SLIMS GATE flow step for starting a new flow execution.

3.5.2. Definition of Steps

After the preamble elements, a number of **<step>** elements are defined. Each **<step>** contains the following definitions:

- **<name>**: *required*. It gives the name to the step.
- **<cannotGoBack>**: *optional*. If set to true, the step will not have the option to return to a previous step in the SLIMS GATE flow.
- **<hidden>**: *optional*. If it is set to true,

```
<hidden>true</hidden>
```

the step calls the **<process>**, but it is not shown in the flow interface. This is useful when no input is required.

- **<input>**: *optional*. Contains a list of **<parameter>**s defining the input to the **<process>**: see Section 3.5.3.
- **<process>**: *required*. It contains the call to a route and/or a validationRoute:

```
<process>
  <validationroute>name-of-validation-route</validationroute>
  <route>name-of-the-route</route>
</process>
```

We will describe the structure of a **<route>**, while the definition of **<validationroute>** is reserved to Beans developers (Section 3.6)

- **<output>**: *optional*. Similar to the <input>, it defines one or more <parameter>s containing the output of the <process>. A parameter defined in this block may be referenced by the <input> of the next step. See Section 3.5.5.

3.5.3. Input Parameters

The <input> block in the <step> element contains one or more <parameter> elements. Required definitions in the <parameter> of an <input> block are <label>, <name> and <type>. A <description> is optional. Parameters of type SingleChoice, MultipleChoice and Configuration require some more definitions, described later.

A <parameter> of the <input> block contains following elements:

- **<label>**: *required*. It indicates the label of the field where input has to be entered (as it looks in the interface).
- **<name>**: *required*. It is the internal name of the variable where the input is stored. If <type> is "File", the <name> needs to be "file" (with lower case f):

```
<parameter>
  <label>Select a file</label>
  <name>file</name>
  <type>File</type>
</parameter>
```

- **<description>**: *optional*. Contains help text for the user to better describe the data expected in this input field.
- **<sameRowAsPrevious>**: This Boolean option can be added in order to have multiple parameters in the same row of a SLIMS GATE form. This option is available for all parameters and can be used to customize the position of parameters in SLIMS GATE forms.
- **<hidden>**: *optional*. If it is set to true,

```
<hidden>true</hidden>
```

the parameter will not be shown to the user.

- **<type>**: *required*. <type>s of interest in the definition of import flows are File (to select a file), MultipleChoice (to select records to be imported), Integer, and Boolean (used in the first step of the flow). In general <type> can be:
 - File: Allows the user to browse the local file system and select a file;
 - Integer: Accepts an integer number;
 - Boolean: A fixed choice between 'yes' or 'no';
 - String: Accepts a String (a one line text);
 - Float: Accepts a float number;
 - Date: Allows to select a date;

- DateTime: Allows to select a date and time;
- Time: Accepts a time in the format HH:MM;
- Password: The entered string is hidden;
- Text: Accepts a long text (more than one line);
- RichText: Accepts a rich text (several lines with bold text, tables...);
- Feedback: Accepts String, long text and rich text but is *not editable*. If the feedback is to be displayed in HyperText Markup Language (HTML), it needs to be wrapped in CDATA:

```
<parameter>
  <name><![CDATA[Feedback]]></name>
  <type>ValueMap</type>
</parameter>
```

- SingleChoice: The user can choose a unique record from a drop-down list. Only the primary key is sent to the input of the following process. See Section 3.5.7;
 - MultipleChoice: The user can choose one or more records from a drop-down list. The all of the records are sent to the input of the following process. See Section 3.5.7;
 - Table: To be used for multiple selection of the same set of parameters. See Section 3.5.9;
 - Configuration: Reserved for Bean developers;
 - SingleChoiceGrid: Grid view of data that are created within a previous bean. It reuses values present in the output of a previous step. See Section 3.5.8;
 - MultipleChoiceGrid: Grid view of data that are created within a previous bean. It reuses values present in the output of a previous step. See Section 3.5.8.
- **<defaultValue>**: *optional*. It defines a default value, for example:

```
<parameter>
  <label>Your age</label>
  <name>age</name>
  <type>Integer</type>
  <defaultValue>6</defaultValue>
</parameter>
```

- **<referencedDefaultValue>**: *optional*. It reuses a value from the output of the previous step. For example if you have following output in a step:

```
<output>
  <parameter>
    <name>outputMap</name>
    <type>ValueMap</type>
  </parameter>
</output>
```

And following input in the next step:

```
<input>
  <parameter>
    <label>Filename</label>
    <name>Filename</name>
    <type>String</type>
    <referencedDefaultValue>outputMap</referencedDefaultValue>
  </parameter>
</input>
```

It will take the default value from the valueMap with name 'outputMap' based on the key 'Filename'

Note that this is only supported for following parameter types: *Boolean, Date, DateTime, Float, Integer, MultipleChoice, SingleChoice, String, Feedback, Text, RichText* and *Time*.

- **<persistent>**: is *optional*. If set to true, this input parameter will be part of the input parameters of all subsequent steps as well.
- **<width>**: is *optional*. It indicates the width of the field.

3.5.4. Process

The process contains the call to a <route> which takes the input parameters, performs some work, and generates an output.

By default the flow waits until the process is terminated (synchronous). A process can be set to asynchronous:

```
<process asynchronous="true">
  <route>name-of-the-asynchronous-route</route>
</process>
```

An asynchronous process runs in background, so the SLIMS interface does not block until its end and is instead released when the process starts. It is useful to set a process to asynchronous if it is the last process of a flow (so its output does not need to be further treated) or if it requires long time, like the import of big Excel files.

Synchronous processes have a default timeout of 60 after which an "Operation timed out" error is sent to the user if the process is not finished. The default timeout can be changed using The parameter 'Timeout' in the process:

```
<!-- Configuring a timeout of 2 minutes -->
<process asynchronous="false" timeout="120000">
  <route>name-of-the-synchronous-route</route>
</process>
```

① The timeout is defined in Milliseconds.

3.5.5. Output Parameters

The <output> receives a message from the process and stores it in a parameter of type <type> and name <name>. This message consists of the *output message* of the last bean called within

the route. The `<type>` needs to match the type returned from the process. The `<name>` can be used to reference this parameter in the following.

The *output message* of the beans described in this chapter is a list of records, corresponding to the `<type>ValueMap</type>`:

```
<output>
  <parameter>
    <name>outputContentMaps</name>
    <type>ValueMap</type>
    <dataSource>Content</dataSource>
    <persistent>true</persistent>
  </parameter>
</output>
```

A `<parameter>` of the `<output>` block contains following elements:

- **`<name>`**: In the following we can use this name as the `<reference>` of a `<valueMap>` (see *choices*).
- **`<type>ValueMap</type>`**: If the output consists in a list of records, we use `ValueMap`;
- **`<dataSource>`**: Is *optional*. If present, it is the name of a table to which the list of records refers: `Content`, `Location`, `Result` and so on. An input single or multiple choice referring to it will look like the *select items* in SLIMS, so if no `<fieldlist>` is specified, the fields with *show in select items* will be shown in the choice.
- **`<persistent>`**: Is *optional*. If set to `true`, this output will be part of the input parameters of all subsequent steps.

A special case consists in a `<parameter>` of `<type> File`:

```
<output>
  <parameter>
    <name>file</name>
    <type>File</type>
  </parameter>
</output>
```

which will suggest a file to download from the browser. In this case the process is expected to generate a `Map` with two entries:

```
fileMap.put("fileName", "nameOfFile.csv");
fileMap.put("bytes", fileContents.getBytes());
```

3.5.6. Flow with Records Selection

If a flow requires a selection of one (or more) content(s), result(s) or attachments, a special parameter can be declared in the `<input>` block. For example:

```
<parameter>
  <label>SLIMS_SELECT_SAMPLES</label>
  <name>SLIMS_SELECT_SAMPLES</name>
```

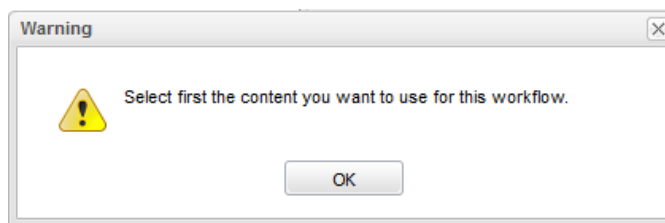
```

<description>SLIMS internal parameter to specify that sample selection is needed for this
flow</description>
<type>StringArray</type>
<required>true</required>
</parameter>

```

where the <label> and the <name> may also be SLIMS_SELECT_ATTACHMENT, SLIMS_SELECT_RESULT, SLIMS_SELECT_ORDERS or SLIMS_SELECT_STABILITY_STUDY. A flow with records selection cannot start if no selection is made, in which case user receives the *warning message*.

Figure 3.6. If a flow requires a selection



The primary keys of selected records are sent to the process.

SLIMS_SELECT_STABILITY_STUDY allows the selection of a record at any level of the tree in the Stability Study Module: a stability study, storage condition, timepoint or timepoint enrollment. Therefore, when the flow actually starts it has the following inputs:

- SLIMS_SELECT_STABILITY_STUDY: The primary key of the selected record in the tree
- SLIMS_SELECT_STABILITY_STUDY_TABLE: The table of the selected record in the tree (either StabilityStudy, StabilityStudyStorage, StabilityStudyTimepoint or TimepointEnrollment)

3.5.7. Parameters of Type SingleChoice and MultipleChoice

The SingleChoice and MultipleChoice types allow the user to make a choice from a list of records:

- A SingleChoice sends only the primary key of the (unique) chosen record to the process, or another key specified by <keyField> ;
- A MultipleChoice sends the whole records to the process.

Note the difference in what is sent to the process in the two cases. They are not interchangeable, so *for the purpose of the Import flows, only MultipleChoice is of interest*.

The list of records to choose is defined by the <valueMap> and can be either a whole table in SLIMS (<table>):

```

<parameter>
<label>Select content</label>
<name>SelectedContent</name>
<type>MultipleChoice</type>
<valueMap>

```



```

    <table>Content</table>
  </valueMap>
</parameter>

```

or the valueMap obtained from the <output> of the previous step (**<reference>**):

```

<parameter>
  <label>Select content</label>
  <name>SelectedContent</name>
  <type>MultipleChoice</type>
  <valueMap>
    <reference>outputContentMaps</reference>
  </valueMap>
</parameter>

```

A valueMap of table type can be filtered to only show records that have a specific value for the given field (*property*), e.g.:

```

<valueMap>
  <table>Content</table>
  <filter>
    <property>cntn_fk_contentType</property>
    <value>cntp_subject</value>
  </filter>
</valueMap>

```

If you would like to do a more complicated filter you can use the "extendedFilter" element:

```

<valueMap>
  <table>Content</table>
  <extendedFilter>cntn_fk_contentType eq 'cntp_subject' and cntn_fk_status eq
'stts_cntn_pending' and cntn_id eq 'S*'</extendedFilter>
</valueMap>

```

The following comparison operators are supported

Table 3.1.

Operator	Description	Example
eq	Equal	cntn_fk_contentType eq 'cntp_subject'
ne	Not equal	cntn_fk_contentType ne 'cntp_subject'
gt	Greater than	cntn_createdOn gt '2024-06-01T00:00:00.000Z'
ge	Greater than or equal	cntn_createdOn ge '2024-06-01T00:00:00.000Z'
lt	Less than	cntn_cf_priority lt 3
le	Less than or equal	cntn_cf_priority le 3
in	In	cntn_fk_contentType in ('cntp_subject', 'cntp_pedigree')

The following logical operators are supported

Table 3.2.

Operator	Description	Example
and	Logical and	cntn_fk_contentType ne 'cntp_subject' and cntn_id eq '0*'
or	Logical or	cntn_fk_contentType ne 'cntp_subject' or cntn_id eq '0*'
not	Logical not	not (cntn_fk_contentType in ('cntp_subject', 'cntp_pedigree'))

Values can be passed like this:

- Numbers should be specified without quotes: cntn_cf_priority eq 3
- Strings should be specified with quotes: cntn_id eq '0000001'
- Strings can contain wildcard characters : cntn_id eq '00*'
- Foreign keys can be referred to with unique identifiers should be specified with quotes: cntn_fk_contentType eq 'cntp_subject'. Wildcards are not supported here
- Dates can be passed using the following date format: '2024-06-01T00:00:00.000Z' (ISO-8601 instant format)

Some more complicated examples:

- rdrc_fk_referenceDataType eq 'store_rdt_countries' and rdrc_name eq 'B*'
- rdrc_fk_referenceDataType eq 'store_rdt_countries' and not (rdrc_name eq 'B*')
- rdrc_fk_referenceDataType eq 'store_rdt_countries' and rdrc_cf_foundedOn gt '1970-01-01T00:00:00.000Z'
- rdrc_fk_referenceDataType eq 'store_rdt_countries' and (rdrc_name eq 'B*' or rdrc_name eq 'S*')

A Single/Multiplechoice can also be defined on a string list, for example:

```
<parameter>
<label>Tables</label>
<name>tableNames</name>
<type>MultipleChoice</type>
<fieldList>
  <entry>Content</entry>
  <entry>ExperimentRun</entry>
</fieldList>
<required>true</required>
</parameter>
```

The choice list can also refer to a *fixed choice* type custom field (**<fixedChoiceCustomField>**), for example:

```
<parameter>
  <label>Select sex</label>
  <name>SelectedSex</name>
  <type>SingleChoice</type>
  <valueMap>
    <fixedChoiceCustomField>cntn_cf_sex</fixedChoiceCustomField>
  </valueMap>
</parameter>
```

In this case, the choice is made as defined in that custom field, which is usually a list of strings (not records). The flow will work only if the custom field cntn_cf_sex is defined.

△ A Single/MultipleChoice on a list of Strings (not Records) sends to the process the String/List of Strings respectively (not Maps).

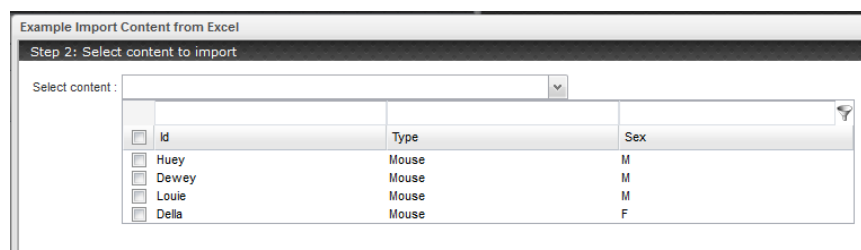
The <input> parameter window can be better configured by the following options:

- **<fieldList>**: Defines the list of fields to show in the dropdown list. For example, importing a list of records where fields Id, Type, and Sex are defined, the fieldList may be defined as follows:

```
<fieldList>
  <entry>Id</entry>
  <entry>Type</entry>
  <entry>Sex</entry>
</fieldList>
```

The dropdown list will look as in Figure 3.7.

Figure 3.7. Definition of <fieldList>



If <fieldList> is not defined:

- For records without dataSource, the <valueField> will be shown in the dropdown list;
- For records with dataSource, the fields with *Show in picklists* checked will be shown, similar to the selection list shown in a SLIMS form.
- **<valueField>**: Specifies which field is shown to the user when he selects a sample. For example:

```
<valueField>Id</valueField>
```

Figure 3.8 illustrates how the menu looks like after having chosen the records named (Id) "Huey" and "Dewey" (note that the Id is shown in the *Select content* field).

Figure 3.8. Definition of <valueField>

Id	Type	Sex
☒ Huey	Mouse	M
☒ Dewey	Mouse	M
☐ Louie	Mouse	M
☐ Della	Mouse	F

- **<required>**: Is optional. If set to true, the flow cannot continue if the user does not make a choice.
- **<defaultValue>**: Is optional and defines a default selection:
 - For type <singleChoice> the defaultValue can be FIRST_ENTRY: the default choice is the first entry in the list;
 - For type <multipleChoice> the defaultValue can be ALL: the default choice is all elements.
- **<keyField>**: Only for <singleChoice>, it defines which field is sent to the process. The primary key (for example cntn_pk) will be sent if keyField is not specified. Note that <multipleChoice> always sends the whole record.

3.5.8. Parameters of Type SingleChoiceGrid and MultipleChoiceGrid

Like the *SingleChoice* and *MultipleChoice* type, the *SingleChoiceGrid* and *MultipleChoiceGrid* respectively allow users to select or to display entries that can be selected and further passed along the flow. Like *SingleChoice* and *MultipleChoice* will show a dropdown to the user, *SingleChoiceGrid* and *MultipleChoiceGrid* will show a grid with checkboxes for selection. Another difference is how the parameter is set. Namely, the *SingleChoiceGrid* or *MultipleChoiceGrid* get instantiated using values that are contained in the output of a previous step.

In the following example, a two-step flow is shown where the first step searches for content and displays the ID, barcode, and location fields. The second step asks the user to select a content record and then displays some feedback:

```
<slimsgate-flow>
  <id>slimsgate-test-flow</id>
  <name>test flow</name>
  <description>A test flow</description>
  <usage>contentmanagement</usage>
  <step>
    <name>Step1 (hidden)</name>
    <hidden>true</hidden>
    <process>
      <route>slimsgate-test-searchContents</route>
    </process>
  </step>
</slimsgate-flow>
```

```

<output>
  <parameter>
    <name>outputContentMaps</name>
    <type>ValueMap</type>
  </parameter>
</output>
</step>
<step>
  <name>Step2 selection of content</name>
  <input>
    <parameter>
      <label>select sample</label>
      <name>selectedSamples</name>
      <description>samplesFromSearchContents</description>
      <type>SingleChoiceGrid</type>
      <valueMap>
        <reference>outputContentMaps</reference>
      </valueMap>
      <fieldList>
        <entry>cntn_Id</entry>
        <entry>cntn_barCode</entry>
        <entry canEdit="true" type="DateTime">cntn_collectionDate</entry>
        <entry>lctn_name</entry>
        <entry>feedback</entry>
      </fieldList>
    </parameter>
  </input>
  <process>
    <route>slimsgate-test-flow</route>
  </process>
</step>
</slimsgate-flow>

```

With such a SLIMS GATE definition, two route builders need to be defined in camel-context. In addition, entries of the types String, Text, Date, DateTime, and Boolean can be edited with the "canEdit" option. Concerning the java part, the FlowBean is a standard java bean that receives a map for the selected entry. The SearchContentBean needs to have List<Map<String, Object>> as the return type of the handler method and the maps must contain the keys listed in the fieldList. Here is an example of such a bean:

```

1
2 package com.genohm.slimsgate.camel.beans.development;
3
4 import java.util.List;
5 import java.util.Map;
6 import org.apache.camel.Handler;
7 import org.springframework.beans.factory.annotation.Autowired;
8 import org.springframework.transaction.annotation.Transactional;
9 import com.genohm.slims.server.repository.queries.ContentRecordQueries;
10 import com.google.common.collect.Lists;
11 import com.google.common.collect.Maps;
12
13 public class SearchContentBean {
14
15     private List<String> feedBackFields;
16     private String feedbackName;
17
18     @Autowired private ContentRecordQueries contentRecordQueries;
19     @Autowired private CustomConfiguration customConfiguration;
20
21     @Transactional(readonly = false)
22     @Handler
23     public List<Map<String, Object>> process() throws Exception {
24         feedBackFields = customConfiguration.getFeedBackFields();
25         feedbackName = customConfiguration.getFeedBackName();

```

```

26 List<Map<String,Object>> outputContentMaps = Lists.newArrayList();
27 List<Map<String,Object>> contents = contentRecordQueries.findByOriginalContent(2562L);
28 for (Map<String,Object> content:contents) {
29     createOutputContentMaps(content, outputContentMaps);
30 }
31 return outputContentMaps;
32 }
33
34 private void createOutputContentMaps(Map<String, Object> content, List<Map<String,
Object>> outputContentMaps) {
35     Map<String,Object> feedBackMap = Maps.newHashMap();
36     for (String feedBackField : feedBackFields) {
37         if (content.containsKey(feedBackField)) {
38             feedBackMap.put(feedBackField, content.get(feedBackField));
39         }
40     }
41     feedBackMap.put(feedbackName, "this is an <br> <u>underlined</u> <br> <b>bold</b> <br>
<i>italic</i> <br> <font color='red'>red</font> <br> feedback");
42     outputContentMaps.add(feedBackMap);
43 }
44 }
45
46

```

Tooltips can also be added to the parameters in the MultipleChoiceGrid. Tooltips were added to the parameter 'icons' in the example below:

```

<parameter>
  <label>Label</label>
  <name>name</name>
  <type>MultipleChoiceGrid</type>
  <valueMap>
    <reference>parametersList</reference>
  </valueMap>
  <fieldList>
    <entry>icons</entry>
  </fieldList>
</parameter>

```

To add a tooltip, the icons and icons_tooltip values have to be added in the bean like in the example below:

```

1
2 for (Map<String, Object> choice : multipleChoices) {
3     ...
4     choice.put("icons", "<img src=\"../images/data_icons/human.gif\">");
5     choice.put("icons_tooltip", "tooltipLine1<br>tooltipLine2");
6 }
7

```

SingleChoice columns can also be added to SingleChoiceGrids or MultipleChoiceGrids, based on options that are provided by a bean. Example in the case of a SingleChoice column in a SingleChoiceGrid:

```

<parameter>
  <name>content</name>
  <label>Content</label>
  <type>SingleChoiceGrid</type>
  <valueMap>
    <reference>contentRecords</reference>
  </valueMap>
  <fieldList>
    <entry>id</entry>
    <entry>volume</entry>
    <entry canEdit="true" type="SingleChoice">choice</entry>
  </fieldList>
</parameter>

```

```
</fieldList>
</parameter>
```

To add the options for the SingleChoice in the grid, the following snippet of code can serve as an example:

```
1
2 List<Map<String, Object>> records = new ArrayList<>();
3 for(int i = 0; i < 10; i++) {
4     Map<String, Object> record = new HashMap<>();
5     record.put("id", "Thing #" + i);
6     record.put("volume", r.nextDouble());
7     record.put("choice", "Default");
8     record.put("choice_options", Lists.newArrayList("Default", "Thing #" + i + "
specific"));
9     records.add(record);
10 }
11 return records;
12
```

Here, note that the 'choice' parameter in slimsgate.xml is filled in by adding 'choice_options' to the HashMap of items in the SingleChoiceGrid.

3.5.9. Parameters of Table Type

An input can be customized in a table of rows, each row consisting of a set of parameters of the types defined above. For example:

```
<input>
<parameter>
<name>Table1</name>
<label>Table1</label>
<type>Table</type>
<parameter>
<label>Boolean field label</label>
<name>BooleanVariable</name>
<type>Boolean</type>
</parameter>
<parameter>
<label>Select the day</label>
<name>Today</name>
<type>Date</type>
</parameter>
</parameter>
</input>
```

corresponding to Figure 3.9

Figure 3.9. <type> Table Input

Boolean field label	Select the day
Yes	18/07/2012
No	

New row

3.5.10. Parameters of Configuration Type

A parameter of *Configuration* type can be used to configure a form with various fields from a single table which will send a map of fields to the process. For example:

```
<parameter>
  <label>Order</label>
  <name>Order</name>
  <description>Create a new order</description>
  <type>Configuration</type>
  <configuration>OrderConfiguration</configuration>
  <hideNonDefaultedFields>true</hideNonDefaultedFields>
  <configurationDefaultValues>
    <foreignKeyEntry key="ordr_fk_orderType" table="OrderType" displayValue="Default Order
Type"/>
    <foreignKeyEntry key="ordr_fk_project" table="Project" displayValue="Default Order
Project"/>
  </configurationDefaultValues>
</parameter>
```

The following elements can be set:

- **<configuration>**: *Required*. The type of records that will be configured. Possible values are: AddressConfiguration, LinkActionConfiguration, OrderConfiguration, RequestConfiguration, ContentConfiguration, and PrintLabelsActionConfiguration.
- **<hideNonDefaultedFields>**: If set to true, only the fields from **<configurationDefaultValues>** will be shown to the user. Otherwise all fields will be shown.
- **<multiplicity>**: This option is only available for ContentConfiguration. The content configuration form will show as many times as the multiplicity specifies. It can be set as an absolute value like this:

```
<multiplicity>
  <value>3</value>
</multiplicity>
```

It can be set as a reference like this. You can refer to a persistent parameter of either the input or output of a previous step.

```
<multiplicity>
  <reference>referenceToAPersistentOutputOrInput</reference>
</multiplicity>
```

- **<configurationDefaultValues>**: To configure default values for various fields. Different types of field entries can be used within one list:
 - **entry**: Has the required argument *key* to indicate which field needs to be shown. It also takes the optional argument *value* for each key to create a default value for the given field.
 - **foreignKeyEntry**: Is used for dynamic choice fields. It has the required arguments *key*, to indicated the field to be filled in, and *table* to specify the SLIMS table. It also takes the optional argument *displayValue* to give a default value per list.

- **referencedEntry**: Takes the data from a previously defined parameter in the SLIMS GATE flow. It has three required arguments. *referencedKey*, which needs the key as it is found in the parameter. *referenceParameter* to indicate which parameter to use. And *key*, the field that will be filled in for the record.

3.6. SLIMS API

In this chapter we present part of the API available for defining Java classes, with particular attention to methods that can be used for fetching data from the database.

3.6.1. SLIMS plugin API overview

- **AddContentWithBarcodeService**: Add a Content with a given barcode instead of having SLIMS generate one.
- **AddLocationWithBarcodeService**: Add a Location with a given barcode instead of having SLIMS generate one.
- **AddRequestService**: Create a Request for a Content, a Requestable, and an Order.
- **ArchiveGeneratorService**: Generate an archive (ZIP) of an ELN entity.
- **AttachmentStore**: Attachment CRUD operations
- **BatchQcResultUpdateService**: Update a QC Result without triggering cascading updates or rule evaluations.
- **BlockPositionService**: (Un-)Block a position using the given Location, row, and column.
- **FilledPositionService**: Calculates the number of filled positions using the given Location.
- **CalculateCellService**: Find the first empty cell of a Location
- **CdsConfig**: Configuration of the Agilent CDS-system (Chromatography Data System).
- **ContentOriginService**: Determine the Content in the start of the run that gave birth to a Content record linked in a step of that run.
- **ConvertRecordService**: Convert records from a human-readable format to the internal format used by SLIMS.
- **DataSourceWrapperProvider**: Give access to the SLIMS meta-information system.
- **DateFormatService**: Format dates.
- **DisplayableEntityService**: Get a more extended representation of an entity or record.
- **ElnService**: Fetch elements of the ELN tree.
- **EmailTemplateService**: Generate e-mails with the Groovy script associated to an e-mail template.
- **EnrollContentInStudyService**: Enroll a Content in a Study.
- **ExperimentRunCreateService**: Create runs.
- **ExperimentRunStepFinishService**: Finish run steps.

- **ExperimentTemplateVersionService**: Version an Protocol.
- **ExportApi**: Fill in a Thymeleaf template with data from a provided Grid template
- **ExportExcelApi**: Export a given Grid template to an Excel sheet given an Excel sheet template.
- **FlagService**: Add/Remove flags on a Content
- **GridTemplateService**: Create a Grid template of chosen type.
- **IlluminaSampleSheetCreateService**: Simplify the creation and customization of Illumina sample sheets.
- **InteractivelImportApi**: Import records from CSV or Excel files.
- **JasperReportService**: Generate Jasper reports.
- **LinkContentService**: Link a Content to an Protocol run step.
- **LinkToQcOrderService**: Add Products to a Content linked to an Order.
- **MacroInspector**: Gives more details about a macro.
- **MacroService**: Executes a macro.
- **MetaTreeCriteriaCompiler**: Compile Meta-Tree criteria.
- **MixContentService**: CRUD operations for mixes.
- **MoveContentsService**: Move Contents.
- **NotificationCreateService**: Create web browser notifications.
- **OlssService**: Get Agilent OpenLab Shared Services (OLSS) elements.
- **OrderRequestableFetchService**: Fetch Requestables matching an Order type.
- **OrderSpecificationFetchService**: Fetch Products matching an Order type.
- **PackageGuidService**: Fetch or add a package GUID for a table and primary key.
- **PedigreeService**: Create pedigree relationships.
- **PopContentByBarcodeService**: Pop Contents and Locations from a Workflow queue via their barcodes.
- **PopFromQueueService**: Low-level service to pop from a Workflow queue.
- **PrintService**: Print labels given a label template, a Content, and more.
- **ProductVersionService**: Version a Product.
- **QcCardRuleEvaluationService**: Evaluate QC rules on a given QC card.
- **QcResultEnricher**: Enrich QC results.
- **QcResultFetchService**: Fetch QC cards results.
- **QuantityService**: Fetch a Quantity, convert a map to a Quantity

- **RReportService**: Generate reports using the R engine.
- **RemotePluginService**: Subscribe to remote plugin heartbeat.
- **ReportService**: Generate reports using the Jasper engine.
- **ReportTemplateVersionService**: Version a Report template.
- **RequeueService**: Requeue and cancel Contents
- **RuleEvaluationService**: Acknowledge rule evaluations.
- **SampleSchedulerInstrumentService**: CRUD operations for Agilent Sample Scheduler instruments for OpenLab analyses.
- **SampleSchedulerService**: CRUD operations for Agilent Sample Scheduler for OpenLab analyses.
- **ScheduleOrderService**: Schedule different kinds of Orders.
- **ScheduleWorkflowService**: Schedule Requests on Workflows.
- **SequenceService**: Generate the next value in a sequence.
- **SignatureService**: Perform a given action without signature or with a provided signature.
- **SlimsVersionService**: Get the version of SLIMS and SLIMS Gate.
- **SopVersionService**: Version an SOP.
- **StabilityVersionService**: Version a Stability Study.
- **TokenApi**: Get the access token of the logged-in user.
- **UnitService**: Convert the abbreviation or primary key of a Unit to the Unit itself.
- **UpdateOrderService**: Change the status of an Order.
- **UserService**: Fetch a User or check if a User is the active User.
- **WorkflowRequestService**: Fetch the Requests on a content in a queue or in an experiment run step.
- **WorkflowSampleSchedulerService**: Interact with Agilent Sample Scheduler for OpenLab sequences.
- **TransactionService**: Wrap code in a new database transaction. Useful when you need to create a sub-transaction, or you are writing code that interacts with the database and may end up being called by code which is not transactional.
- **ContentTypeMoveCategoryService**: Move Content type in other category.

3.6.2. Database Interaction Example

In this section we illustrate common interactions with the database through an example. These interactions are:

1. Fetching records in database;
2. Updating an existing record;

3. Creating a new record;

4. Deleting a record.

The example contains comments and further comments are added after it.

```
1
2 package com.genohm.slimsgate.camel.beans.development;
3
4 import java.util.List;
5 import java.util.Map;
6
7 import org.apache.camel.Handler;
8 import org.apache.camel.Header;
9 import org.slf4j.Logger;
10 import org.springframework.beans.factory.annotation.Autowired;
11 import org.springframework.transaction.annotation.Transactional;
12
13 import com.genohm.slims.common.model.Content;
14 import com.genohm.slims.common.model.ContentMeta;
15 import com.genohm.slims.common.model.ContentType;
16 import com.genohm.slims.common.model.ContentTypeMeta;
17 import com.genohm.slims.common.model.ExperimentRunStepContent;
18 import com.genohm.slims.common.model.Location;
19 import com.genohm.slims.common.slimsgate.SlimsgateParameter;
20 import com.genohm.slims.common.util.StringUtil;
21 import com.genohm.slims.server.dao.common.Dao;
22 import com.genohm.slims.server.dao.criterion.SlimsRestrictions;
23 import com.genohm.slims.server.dao.request.FetchRequest;
24 import com.genohm.slims.server.dao.request.FetchRequestImpl;
25 import com.genohm.slims.server.dao.response.FetchResponse;
26 import com.genohm.slims.server.repository.queries.ContentRecordQueries;
27 import com.genohm.slims.server.repository.queries.ContentTypeRecordQueries;
28 import com.genohm.slimsgate.camel.gatekeeper.SlimsGateKeeper;
29 import com.genohm.slimsgate.camel.gatekeeper.SlimsProxy;
30 import com.genohm.slimsgateclient.workflow.SlimsFlowInitParam;
31 import com.google.common.base.Optional;
32 import com.google.common.collect.Maps;
33
34 public class SlimsdaoShowcaseBean {
35
36     @Autowired
37     private Dao<Content> contentDao;
38     @Autowired
39     private Dao<ContentType> contentTypeDao;
40     @Autowired
41     private Dao<ExperimentRunStepContent> experimentRunStepContentDao;
42
43     @Autowired
44     private ContentRecordQueries contentRecordQueries;
45     @Autowired
46     private ContentTypeRecordQueries contentTypeRecordQueries;
47     @Autowired
48     private LocationRecordQueries locationRecordQueries;
49     @Autowired
50     private StatusQueries statusQueries;
51
52     @Transactional(readOnly = false)
53     @Handler
54     public void run(@Header(SlimsGateKeeper.SLIMS_WORKFLOW_INIT_PARAMETER)
55 SlimsFlowInitParam initParam,
56 @Header(SlimsGateKeeper.SLIMS_PROXY) SlimsProxy slimsproxy) {
57     Logger log = slimsproxy.getLogger(getClass());
58
59     //This log sends logs to
60     // - the client running this flow,
```

```

61 // - the database (to view the logging later)
62 // - and to the normal logging files of SLIMS GATE
63 log.info("Starting slimsdao showcase");
64
65 //input parameter values (<input> block in the flow definition)
66 Map<String, Object> valueMap = initParam.getInputParameterValues();
67 //get the selected content in SLIMS (the parameter named SLIMS_SELECT_SAMPLES has to be
68 //set in SLIMS GATE definition!)
69 Long[] contentPks =
70 StringUtil.getAsLongArray(valueMap.get(SlimsgateParameter.SLIMS_SELECT_SAMPLES));
71 // get the experiment run step pk using initParam.getExperimentRunStepId()
72 // For "default" protocols: the pk of the step where the flow was executed
73 // For "simplified" protocols:
74 // If NOT executed in an Action block, the experiment run step pk of the block the
75 // flow was started from
76 // If it is in an Action block, then you will get the pk of the page you are in
77 // Note that you would have access to other PKs:
78 // pk of the Action block with
79 'initParam.getInputParameterValues().get(SlimsgateParameter.SLIMS_ACTION_BLOCK) '
80 // pk of the Input block with
81 'initParam.getInputParameterValues().get(SlimsgateParameter.SLIMS_INPUT_BLOCK) '
82 // pk of the Output block with
83 'initParam.getInputParameterValues().get(SlimsgateParameter.SLIMS_OUTPUT_BLOCK) '
84 Long experimentRunStepPk = StringUtil.getAsLong(initParam.getExperimentRunStepId());
85
86 Long contentRemovedStatusPk =
87 statusQueries.findByUniqueIdentifier("stts_cntn_removed").toJavaUtil().orElseThrow().getStts_pk();
88
89
90 //1. Fetching database:
91 //One step fetch: you want to get a Record (== Map<String, Object>) containing ALL
92 //values: default fields, custom fields, and display values
93 //Find a record by primary key
94 Optional<Map<String, Object>> recordLocation = locationRecordQueries.findByPk(45L);
95
96 //Note the use of Optional here; a contenttype with the name "Mouse" might not exist
97 //Find a record by name
98 Optional<Map<String, Object>> recordContentType =
99 contentTypeRecordQueries.findByName("Mouse");
100 //Find a content by barcode
101 Optional<Map<String, Object>> recordContent =
102 contentRecordQueries.findByBarCode("00000546");
103
104 //Find the content in an experimentrunstep
105 List<Map<String, Object>> byExperimentRunStep =
106 contentRecordQueries.findByExperimentRunStep(experimentRunStepPk);
107
108 //General fetching: to be used if the wanted findBy... is not available or if you want
109 //to combine many criteria:
110 //For example: Fetch all content that is selected in SLIMS with quantity > 5
111 //First make a fetch request
112 FetchRequest fetchRequest = new FetchRequestImpl()
113     .setCriterion(SlimsRestrictions.and()
114         .add(SlimsRestrictions.equals(ContentMeta.FK_CONTENT_TYPE, 5L))
115         .add(SlimsRestrictions.in(ContentMeta.PK, contentPks)));
116
117 //Execute this request to get a response
118 FetchResponse fetchResponse = contentDao.fetch(fetchRequest);
119
120 //The total result list is available here
121 List<Map<String, Object>> contentRecords = fetchResponse.getResultList();
122
123 //This is also possible using the recordQueries to make it a bit shorter
124 List<Map<String, Object>> sameContentRecords =
125     contentRecordQueries.fetch(SlimsRestrictions.and()
126         .add(SlimsRestrictions.equals(ContentMeta.FK_CONTENT_TYPE, 5L))
127         .add(SlimsRestrictions.in(ContentMeta.PK, contentPks)));

```

```

118
119
120 //2. Updating Content Records
121 for (Map<String, Object> contentRecord: contentRecords) {
122     //Update all the found contentRecords
123     contentRecord.put(ContentMeta.SHORT_ID, "ABCD");
124     contentRecord.put(ContentMeta.FK_STATUS, contentRemovedStatusPk);
125     Map<String, Object> updatedRecord = contentDao.update(contentRecord);
126     //We could continue working with this updatedRecord here...
127 }
128
129 //3. Add a new content record
130 Map<String, Object> contentRecordToAdd = Maps.newHashMap();
131 contentRecordToAdd.put(ContentMeta.FK_STATUS, 28L); //REQUIRED
132 contentRecordToAdd.put(ContentMeta.FK_CONTENT_TYPE,
    contentRecordQueries.findByName("Blood").get().get(ContentTypeMeta.PK)); //REQUIRED
133 contentRecordToAdd.put(ContentMeta.ID, "Demo id");
134 contentRecordToAdd.put("cntn_cf_naField"+ DaoConstants.IS_NA, true);
135 Map<String, Object> addedRecord = contentDao.add(contentRecordToAdd);
136
137 //4. Remove a record (will remove it completely from the database)
138 contentDao.remove(addedRecord);
139 }
140 }
141

```

Comments:

- **Line 36:** `@Autowired`: springframework annotation: see the Spring manual
- **Line 37:** `Dao<Table>` objects: contain methods for the four basic interactions on one table: fetching, updating, adding, and removing. For example `Dao<Content>` (called `contentDao`) can be used to fetch and manipulate content records. In the same way you can use `Dao<Location>` to interact with Location records.
- **Line 46:** `TableRecordQueries` like `ContentRecordQueries` or `ContentTypeRecordQueries`. Contain methods that allow for an easy way of fetching data. The predefined `findBy` methods are for cases that are often useful; for example: `contentRecordQueries.findByBarcode()` or `locationRecordQueries.findByParent()`. Note that these fetches are always filtered on the access rights of the user executing the SLIMS GATE Flow. If a user doesn't have access to a certain contentRecord, then neither `contentDao.fetch` nor the `contentRecordQueries` methods will ever return this content record.
- **Line 50:** `@Transactional` an annotation NEEDED if the method has to interface with the database
- **Line 51:** `@Handler` marks the method as the one that will be executed by camel (if no other method is specified)
- **Line 55:** This is a special Logger that allows a developer to let his logging be saved in the database and be sent to the user executing the SLIMS GATE flow
- **Line 64:** This is how the primary keys of selected samples can be retrieved. This line will work only if the input parameter `SLIMS_SELECT_SAMPLES` is set in the SLIMS GATE flow definition (see Section 3.5.6):

```

<input>
<parameter>

```

```
<label>SLIMS_SELECT_SAMPLES</label>
<name>SLIMS_SELECT_SAMPLES</name>
<type>StringArray</type>
</parameter>
</input>
```

- **Line 66:** Converting to the right type: Note the use of `com.genohm.slims.common.util.StringUtil`. `StringUtil` offers a fairly safe way of converting simple types in a fairly robust fashion. For example `StringUtil.getAsInteger()` can deal with the fact that the argument is a Long, Integer, or String.
- **Line 88:** A `FetchRequest` object contains a request for the database. A fetching criterion is set by `setCriterion(SlimsCriterion)` and the `SlimsCriterion` is defined by a `SlimsRestrictions` method. In this example we encounter two types of restrictions:
 - Two simple restrictions with two arguments: `gt` (greater than) and `in` (included);
 - A junction of restrictions, realized by the `and()` (or() exists as well) and `add()` methods.
 - ① `TableRecordQueries` can be more effectively used for fetching the database.
- **Line 90:** `ContentMeta.CONSTANTS` is a class that collects the names of default fields as constants. For example `ContentMeta.ID = "cntn_id"`. It is useful in queries (see following).
- **Line 90:** `SlimsRestrictions.equals(ContentMeta.FK_CONTENT_TYPE, 5L)` returns records with field `"cntn_fk_contentType"` equal to 5L.
- **Line 101:** Same fetch but with `contentRecordQueries`.
- **Line 119:** The status is required when adding a content. If the status is not pending, removed, or cancelled, the location will also be required.
- **Line 120:** The content type is required when adding a content.
- **Line 122:** The field `cntn_cf_naField` is a field that can be NA. The value in this case is set to NA.
- **Line 123:** Add: note that this function assigns a new primary key to the new record. All required fields need to be set before adding the content (`cntn_fk_status`, `cntn_fk_contentType`, custom fields which are required, location if status requires it, and barcode if not generated by SLIMS).
- **Line 126:** Removes a record permanently from the database. This action should be used with caution.

Note that this example never refers to a specific user. This means that the user executing the flow will be used to create history, fill in audit fields, etc. If no user can be determined (for example in a SLIMS GATE flow that runs every night automatically), you can use `ActiveUser.set()` at the top of your code. Database interactions are not allowed without an active user.

3.6.3. Database Interaction Objects

`Dao<Table>` objects: Contain methods for the four basic interactions on one table: fetching, updating, adding, and removing.

3.6.3.1. Methods Summary

Modifier and Type	Method and Description
<code>Map<String, Object></code>	<code>add(Map<String, Object> newValues)</code> Adds a new record to a table. Returns the newly added record.
<code>Map<String, Object></code>	<code>update(Map<String, Object> newValues)</code> Updates a record from a table. Returns the newly updated record.
<code>void</code>	<code>remove(Map<String, Object> values)</code> Removes a record from a table.
<code>long</code>	<code>count(FetchRequest fetchRequest)</code> Counts the records that match the criteria in a fetchrequest. Returns the amount of records that match the fetchrequest.
<code>FetchResponse</code>	<code>fetch(FetchRequest fetchRequest)</code> Fetches the records that match a fetchrequest. Returns a fetchresponse describing the records.
	①Consider using <code>TableRecordQueries</code> instead.

It is also possible to use the `ExtraFieldValidationBean` which performs extra validation such as checking the value of a dynamic choice field.

The two following methods enable throwing errors if a non-existing field is used:

`validate(Map<String, Object> record, String tableName)` and `validateContent(Map<String, Object> record)`. The first is generic whereas the second is specific for contents. Both are throwing `InvalidFieldException` if a field in the map does not exist or `InvalidDynamicChoiceValueException` if a dynamic field has a value that does not exist in the referenced table.

The following DAO are not available in the API:

- Configuration
- Changelog
- Datatype
- Dimension
- ImportedPackage
- PackageInstallLog
- Slims

- SlimsApiClient
- SlimsPlugin
- UserGroup

3.6.4. Database Queries Objects

3.6.4.1. *TableRecordQueries*

All tables in the SLIMS database have a corresponding *TableRecordQueries* object. Such objects contain methods that provide an easy way to fetch data. For example:

```
ContentRecordQueries  
LocationRecordQueries
```

Some `findBy` methods are predefined for the most often useful cases. For example: `contentRecordQueries.findByBarcode()` or `locationRecordQueries.findByParent()`. Note that these fetches are always filtered on the access rights of the user executing the SLIMS GATE Flow. If a user doesn't have access to a certain contentRecord then neither `contentDao.fetch` nor the `contentRecordQueries` methods will ever return this content record.

The `return` type of these methods is a `Map` containing all fields (default and custom) of the found record(s).

3.6.4.2. *TableQueries*

The *TableQueries* object has the same role as Section 3.6.4.1 with the difference that the `return` type of the methods is a `Java Object`. For example:

```
Content  
Location
```

These objects have the advantage of being smaller than `Maps`, but they only contain information about default fields.

3.6.4.3. *UserQueries*

A special object containing fetch methods dedicated to the `User` table. For example:

```
UserQueries userQueries;  
List<User> userQueries.findByUserName(String userName, boolean checkActive);
```

3.6.4.4. *AttachmentStore*

The *AttachmentStore* object contains a few useful methods to interact with the repository. In particular, an `InputStream` object containing the file contents can be obtained as follows:

```
@Autowired  
private AttachmentStore attachmentStore;
```

```
...
    long attachmentPk = 2L;
    attachmentStore.getFileContents(attachmentPk);
```

AttachmentStore can also be used to add attachments. This can be done the following way:

```
@Autowired
private AttachmentStore attachmentStore;
...

byte[] fileBytes = "I am file content".getBytes();
Attachment attachment = new Attachment();
attachment.setAttm_name("demoFile.txt");
// To link the attachment to a content:
AttachmentLink link = new AttachmentLink(DaoConstants.CONTENT, 42L);
// Or to link the attachment to an experiment run step:
// AttachmentLink attmLink = new AttachmentLink(DaoConstants.EXPERIMENT_RUN_STEP, xprs_pk);
attachmentStore.addAttachment(attachment, Arrays.asList(link), new
ByteArrayInputStream(fileBytes));
```

An attachment can be linked to any table and record using AttachmentLink. Multiple links can be created to link the attachment to more than one record. It is possible to add an attachment to the SLIMS repository without attaching it to anything using null instead of Arrays.asList(link).

3.6.5. SlimsCriterion to narrow down FetchRequests

A FetchRequest object contains a request for the database. Setting criteria for fetching is done with setCriterion(SlimsCriterion) and the SlimsCriterion is defined by a SlimsRestrictions method.

Examples of SlimsRestrictions:

1. SlimsRestrictions.equals(String fieldName, Object value)

With this criterion, the FetchRequest will only return records that have a field with fieldName that has the set value. Example:

```
SlimsRestrictions.equals("cntn_cf_patientNumber", "0001234")
```

2. SlimsRestrictions.between(String fieldName, Object start, Object end)

With this criterion, the FetchRequest will only return records that have a field with fieldName that has a date between the set start and end. Use SlimsRelativeDate to define the start and end. Example: SlimsRestrictions.between(ContentMeta.COLLECTION_DATE, SlimsRelativeDate.START_OF_MONTH, SlimsRelativeDate.END_OF_MONTH)

3.6.6. Setting the ActiveUser

Database interactions are not allowed without an active user. The active user by default is the user executing the flow and will be used to create history, fill in audit fields, etc.

If no user can be determined (for example in a SLIMS GATE flow that runs every night automatically), the runAsActiveUser() can be used like this:

```
ActiveUser.runAsActiveUser(user, () -> {
    //do stuff here
})
```

```
}}
```

The `ActiveUser.set()` should not be used anymore.

3.6.7. SlimsGateErrorException

`SlimsGateErrorException` is the exception to be thrown if a user-friendly error message is needed.

3.6.8. Checking for Flow Interruption

A flow can be set to be `interruptable`. The interruption action can be checked in the code with the method:

```
slimsproxy.checkForInterruption();
```

3.6.9. Working with Progress Bar

Whenever the user is interested in the progress of a flow, a progress bar can be added by adding the `showProgress` element in the `slmsgate.xml` file and in the bean itself. The level of the progress bar is managed with the used beans in a SLIMS GATE flow in two ways:

- By letting `slimsproxy` compute the increase: `slimsproxy.increaseProgress()`;
- By setting the progress at each step: `slimsproxy.setProgress(currentIndex, total, "messageUnderProgressBar")`;

3.6.10. Trigger Experiment Run Step Refresh via SLIMS GATE

An `ELN_FINISH_FLOW` event is fired when the user closes the wizard window after running the SLIMS GATE flow (analogous to the existing `ELN_START_FLOW` which was used to start the SLIMS GATE flow in the first place). The `ELN_FINISH_FLOW` event triggers a (lazy) refresh of all tabs: In which the tab is refreshed when it's selected by the user instead of being instantly refreshed. For the Detail tab (`ExperimentRunStepFormTab`): An explicit refresh of the record is present. For the `DynamicResults` tabs: Explicit reload of dynamic results is present.

3.6.11. Use NGS Samplesheet Generation API in Plugins

The `IlluminaSampleSheetCreateService` interface can be accessed for engineers to develop custom SLIMS plugins that generate samplesheets for Illumina sequencers. Refer to the inline javadoc in the codebase for steps to implement Illumina Sample Sheet Generation API with plugins. Look for "`IlluminaSampleSheetCreateServiceInterface`." The basic steps are as follows:

1. Create a SLIMS GATE flow with "workflow" usage. It must contain a single hidden step with a route defined. This will allow the SLIMS GATE Flow to be available in Illumina sample sheet protocol step configuration.

2. The route output must be a string which will be the content of the sample sheet.
3. Use `IlluminaSampleSheetCreateService::generateSampleSheetModel` to get a default model of the sample sheet.
4. Modify the model as needed.
5. Use `IlluminaSampleSheetCreateService::generateContentFromModel` to validate and generate the sample sheet content from the model.

3.6.12. Create Notifications via SLIMS GATE

Notifications in SLIMS can be created via a SLIMS GATE flow. They can be created using either the `slimsproxy`:

```
slimsproxy.notify(String NotificationTitle, String NotificationContentText,
Collection<Long> usersToNotifyPk);
```

or the `notificationCreateService`:

```
createNotification(String NotificationTitle, String NotificationContentText,
Collection<Long> usersToNotifyPk, User user);
```

The notifications will be accessible in SLIMS from the top-right menu bar under the bell menu which will turn red when the user has an unread notification.

3.6.13. Using TokenApi

When using OAuth as the authentication method for users in SLIMS, the authentication token can be used to authenticate against external systems. The `TokenApi` can be used to get the access token of the user that is logged in. As an example: the following bean uses the access token for SLIMS to access an account on a website "<https://dev-xxxxxxx.oktapreview.com/user-info>", getting some information about the user on this system.

```
package com.genohm.slims.custom.beans;

import java.io.BufferedReader;
import java.io.IOException;
import java.io.InputStreamReader;
import java.net.HttpURLConnection;
import java.net.URL;

import org.apache.camel.Handler;
import org.apache.camel.Header;
import org.slf4j.Logger;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Component;
import org.springframework.transaction.annotation.Transactional;

import com.genohm.slims.common.model.User;
import com.genohm.slims.server.service.security.TokenApi;
import com.genohm.slims.server.service.user.UserService;
import com.genohm.slimsgate.camel.gatekeeper.SlimsGateKeeperConstants;
```

```

import com.genohm.slimsgate.camel.gatekeeper.SlimsProxy;
import com.genohm.slimsgateclient.workflow.SlimsFlowInitParam;

@Component
public class SimpleIntegrationBean {

    @Autowired
    private TokenApi tokenApi;
    @Autowired
    private UserService userService;

    @Transactional
    @Handler
    public void test(
        @Header(SlimsGateKeeperConstants.SLIMS_WORKFLOW_INIT_PARAMETER) SlimsFlowInitParam
        initParam,
        @Header(SlimsGateKeeperConstants.SLIMS_PROXY) SlimsProxy slimsProxy) {
        User user = userService.findByName(initParam.getUserName());
        Logger logger = slimsProxy.getLogger(getClass());
        logger.info("Getting okta user info for " + user.getUser_username());
        tokenApi.getAccessToken().ifPresent(accessToken -> {
            try {
                URL url = new URL("https://dev-xxxxxxx.oktapreview.com/userinfo");
                HttpURLConnection http = (HttpURLConnection)url.openConnection();
                http.setRequestProperty("Accept", "application/json");
                http.setRequestProperty("Authorization", "Bearer " + accessToken);
                http.setRequestMethod("GET");
                logger.info("Response code " + http.getResponseCode());
                StringBuilder result = new StringBuilder();
                try (BufferedReader reader = new BufferedReader(new
                    InputStreamReader(http.getInputStream()))) {
                    for (String line; (line = reader.readLine()) != null; ) {
                        result.append(line);
                    }
                }
                logger.info("User info: ");
                logger.info(result.toString());
                http.disconnect();
            } catch (IOException e) {
                e.printStackTrace();
            }
        });
    }
}

```

3.6.14. Updating Quantities

It is possible to explicitly add or update a quantity in a record. This is a best practice example of how to do so:

```

...
// units have to be retrieved by their SLIMS abbreviation, not by name.
// using the UnitService to find the unit by abbreviation and directly get this as a Unit
// object
Unit unit = unitService.convertToUnit("m");
Quantity quantity = new Quantity("0.1111", unit);

//Do not use doubles or other datatypes that are not exact for the amount. Use Strings
// instead!
myContentRecord.put("cntn_cf_length", quantity);
contentDao.update(myContentRecord);

```

The QuantityService API can be used to work more easily with the objects or records returned from database requests. Available conversions methods are:

- **Converting a map to Quantity:** `Quantity convertMapToQuantity(Map<String, Object> quantityMap);`
- **Converting an object to Quantity:** `Quantity castOrConvertToQuantity(Object quantity);`

Example of usage:

```
@Autowired
private QuantityService quantityService;
...
// get quantity record with pk 5 and add it to the length value of myContentRecord
Map<String, Object> quantityRecord = quantityService.getQuantityRecord(5);
Quantity quantityObject = quantityService.convertMapToQuantity(quantityRecord);
Quantity contentQuantity =
    quantityService.castOrConvertToQuantity(myContentRecord.get("cntn_cf_length"));
myContentRecord.put("cntn_cf_length", contentQuantity.add(quantityObject));
contentDao.update(myContentRecord);
```

3.6.15. Enrolling content in a study

A content can be enrolled in a study with the `EnrollContentInStudyService` which provides the following methods:

Modifier and Type	Method and Description
void	<code>enroll(Long contentPk, Long studyPk)</code> Enrolls the content in the study. The enrollment is done by the currently logged-in user. Checks if the content can be enrolled in the study, preventing enrolling it twice in the same study.
void	<code>enroll(Long contentPk, Long studyPk, User user)</code> Enrolls the content in the study. The enrollment is done by the given user. Checks if the content can be enrolled in the study, preventing enrolling it twice in the same study.

3.6.16. Short Examples

3.6.16.1. Linking Content to Experiment Run Step

```
...
Map<String, Object> valueMap = initParam.getInputParameterValues(); // SlimsFlowInitParam
initParam
Map<String, Object> newExperimentRunStepContent = new HashMap<>();
newExperimentRunStepContent.put(ExperimentRunStepContentMeta.FK_CONTENT,
    contentMap.get("cntn_pk")); // Map<String, Object> contentMap ...
newExperimentRunStepContent.put(ExperimentRunStepContentMeta.FK_EXPERIMENT_RUN_STEP,
    valueMap.get(SlimsgateParameter.SLIMS_EXPERIMENT_RUN_STEP));
newExperimentRunStepContent.put(ExperimentRunStepContentMeta.USAGE,
    ExperimentRunStepContent.USAGE_INPUT);
```

```
experimentRunStepContentDao.add(newExperimentRunStepContent);           // private
    Dao<ExperimentRunStepContent> experimentRunStepContentDao;
```

3.6.16.2. Create a Usage Instance

The following class creates a Usage Instance for a content linked to an experiment run step.

```
package com.genohm.slimsgate.test;           // The package needs to be changed accordingly

import java.util.HashMap;
import java.util.List;
import java.util.Map;

import org.apache.camel.Handler;
import org.apache.camel.Header;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.transaction.annotation.Transactional;

import com.genohm.slims.common.model.ExperimentRunStepContentUsage;
import com.genohm.slims.common.model.XprsContentUsageInstance;
import com.genohm.slims.server.dao.common.Dao;
import com.genohm.slims.server.repository.queries.XprsContentUsageInstanceRecordQueries;
import com.genohm.slimsgate.camel.gatekeeper.SlimsGateKeeper;
import com.genohm.slimsgate.camel.gatekeeper.SlimsProxy;
import com.genohm.slimsgateclient.workflow.SlimsFlowInitParam;

public class TestUsageInstanceBean {

    @Autowired
    private Dao<XprsContentUsageInstance> xprsContentUsageInstanceDao;
    @Autowired
    private XprsContentUsageInstanceRecordQueries xprsContentUsageInstanceRecordQueries;
    @Autowired
    private Dao<ExperimentRunStepContentUsage> experimentRunStepContentUsageDao;

    @Transactional(readOnly = false)
    @Handler
    public void process(@Header(SlimsGateKeeper.SLIMS_WORKFLOW_INIT_PARAMETER)
        SlimsFlowInitParam initParam,
        @Header(SlimsGateKeeper.SLIMS_PROXY) SlimsProxy slimsproxy) {

        List<Map<String, Object>> usages =
            xprsContentUsageInstanceRecordQueries.findByRunStep(initParam.getExperimentRunStepId());
        // Create Instance
        Map<String, Object> newUsageInstance = new HashMap<String, Object>();
        newUsageInstance.put("xcui_fk_xRunStep", initParam.getExperimentRunStepId());
        newUsageInstance.put("xcui_seqNo", 4);           // Sequence number of the
        instance. Needs to be an Integer.
        Map<String, Object> createdInstance = xprsContentUsageInstanceDao.add(newUsageInstance);

        Map<String, Object> newQuantityMap = new HashMap<String, Object>();
        newQuantityMap.put("amount", "5");           // Note that the amount is a
        String
        newQuantityMap.put("unit_pk", 6);

        Map<String, Object> newUsage = new HashMap<String, Object>();
        newUsage.put("xscu_fk_xRunStepContent", 2L);           // This primary key may be
        retrieved using ExperimentRunStepContentQueries
        newUsage.put("xscu_fk_usedQuantity", newQuantityMap);
        newUsage.put("xscu_fk_xprscUsageInstance", createdInstance.get("xcui_pk"));

        experimentRunStepContentUsageDao.add(newUsage);
    }
}
```

⚠ There is currently no method to automatically calculate the next sequence number.

3.6.16.3. Generate a Jasper PDF Report Using Jasper API

The following class creates a Jasper PDF Report based on an XML document.

```
import java.io.ByteArrayOutputStream;
import java.util.HashMap;
import java.util.Map;

import net.sf.jasperreports.engine.JasperFillManager;
import net.sf.jasperreports.engine.JasperPrint;
import net.sf.jasperreports.engine.data.JRXmlDataSource;
import net.sf.jasperreports.engine.export.JRPdfExporter;
import net.sf.jasperreports.export.SimpleExporterInput;
import net.sf.jasperreports.export.SimpleOutputStreamExporterOutput;

import org.dom4j.Document;
import org.dom4j.DocumentHelper;
import org.dom4j.io.DOMWriter;

public class JasperReportBean {

    public void generateJasperReport() throws Exception {
        //Parameters
        String selectExpression = "/";
        String pathToJasperTemplate = "conf/META-INF/jasper/report.jrxml";
        Map<String, Object> fillReportParams = new HashMap<>();

        //Report output stream
        ByteArrayOutputStream reportOutputStream = new ByteArrayOutputStream();

        //XML document
        Document xmlDocument = DocumentHelper.createDocument();

        // ... here the XML document should be filled ...

        //XML dataSource
        DOMWriter domWriter = new DOMWriter();
        org.w3c.dom.Document w3cDocument = domWriter.write(xmlDocument);
        JRXmlDataSource xmlDataSource = new JRXmlDataSource(w3cDocument, selectExpression);

        JasperPrint jasperPrint = JasperFillManager.fillReport(pathToJasperTemplate,
            fillReportParams, xmlDataSource);

        JRPdfExporter jrpdfExporter = new JRPdfExporter();
        jrpdfExporter.setExporterInput(new SimpleExporterInput(jasperPrint));
        jrpdfExporter.setExporterOutput(new SimpleOutputStreamExporterOutput(reportOutputStream));
        jrpdfExporter.exportReport();

        byte[] reportBytes = reportOutputStream.toByteArray();
    }
}
```

3.6.16.4. Trigger Generation of Core Report in SLIMS GATE

```
...
//Example class for modifying a core report XML, when built-in metatree is not enough.
public class WrappedCoreReportBean {
    @Autowired
    private ReportService reportService;
    @Autowired
    private MetaTreeCriteriaCompiler metaTreeCriteriaCompiler;

    @Transactional(readOnly = false)
    @Handler
```



```

public void run(@Header(SlimsGateKeeper.SLIMS_WORKFLOW_INIT_PARAMETER) SlimsFlowInitParam
initParam,
    @Header(SlimsGateKeeper.SLIMS_PROXY) SlimsProxy slimsproxy) throws Exception {

    Logger log = slimsproxy.getLogger(getClass());

    log.info("Generate XML from core template");
    Long reportTemplatePk = 17L; // In a production bean report template pk should be managed differently
    List<Long> pkContents =
Arrays.asList( StringUtil.getAsLongArray( initParam.getInputParameterValues().get(SlimsgateParameter.SLIMS_S
Document coreXml = generateCoreXml(reportTemplatePk, pkContents, ActiveUser.get());

    log.info("modify XML");
    modifyXml(coreXml);

    log.info("compiling report");
    Set<Long> longSet = Sets.newHashSet(pkContents);
    ByteArrayOutputStream byteOut = new ByteArrayOutputStream();
    reportService.generateReport(coreXml, reportTemplatePk, Optional.of(longSet),
ExportFormat.PDF, byteOut, DevNullFeedback.INSTANCE, ActiveUser.get());

}

private Document generateCoreXml(Long reportTemplatePk, List<Long> pkContents, User
user) throws Exception {
    // This works is only one content is selected.
    // In a production bean, one should deal with the multi-selection case.
    // That depends also on the kind of core report (multi-select or right click).
    Optional<SlimsCriterion> criterion = Optional.of(SlimsRestrictions.equals(ContentMeta.PK,
pkContents.get(0)));
    Map<Long, SlimsCriterion> criterionMap =
metaTreeCriteriaCompiler.compileCriterionMap(reportTemplatePk, user);
    return reportService.generateXml(reportTemplatePk, criterion, criterionMap,
DevNullFeedback.INSTANCE, user);
}

private void modifyXml(Document xml){
    ...
}

```

3.7. SLIMS GATE flows and routes using the SLIMS REST API

3.7.1. Introduction

In this chapter we explain how SLIMS GATE flows and routes can be started using the SLIMS REST API.

It is no longer advised to use custom Restlet routes in SLIMS GATE Plugins after SLIMS 6.7.2 for the following reasons:

- Restlet is deprecated and has been removed in SLIMS 6.8. Any Restlet route will need to be converted to SLIMS REST routes or use other Camel mechanisms.
- Restlet routes do not have associated authentication.
- Restlet routes require customization on Apache HTTPD. This is not required here.
- SLIMS Agilent Hosted (Cloud) does not allow Restlet routes

All available rest endpoints for starting SLIMS GATE flows and routes are documented in the SLIMS REST API documentation.

3.7.2. Exposing a SLIMS GATE route through SLIMS REST

3.7.2.1. Writing the plugin

If you want to expose a SLIMS GATE route through SLIMS REST, the Route ID you want to call needs to start with "slimsrest-". The route needs to return a SlimsgateRestResponse. For example the following route is exposed to SLIMS GATE:

```
@Autowired
private SayHelloBean sayHelloBean;

@Component
public class SayHelloBuilder extends RouteBuilder {

    @Override
    public void configure() throws Exception {
        from("direct:hello")
            .bean(sayHelloBean)
            .routeId("slimsrest-sayHello");
    }
}
```

With the following bean:

```
@Component
public class SayHello {

    @Handler
    public SlimsgateRestResponse test(
        @Header(SlimsGateKeeperConstants.SLIMS_WORKFLOW_INIT_PARAMETER) SlimsFlowInitParam
        slimsFlowInitParam) {

        return new SlimsgateRestResponse("application/text", "slimsgate says hi");
    }
}
```

The SlimsgateRestResponse takes a HTTP header content type and a body. The list of available content types can be found here: <https://docs.spring.io/spring-framework/docs/current/javadoc-api/org/springframework/http/MediaType.html>

This can now be called through SLIMS REST:

```
curl -u user:pass http://url/slims/slimsrest/v1/rest/slimsgate/route/
slimsrest-sayHello
```

And will result in the following response:

```
slimsgate says hi
```

Returning a json body is also possible:

```
@Handler
public SlimsgateRestResponse test(
```

```
@Header(SlimsGateKeeperConstants.SLIMS_WORKFLOW_INIT_PARAMETER) SlimsFlowInitParam
slimsFlowInitParam)
    throws JsonProcessingException {

    return new SlimsgateRestResponse("application/json",
        new ObjectMapper().writeValueAsString(new DemoObject("value1", "value2")));
}

public class DemoObject {
    private final String property1;
    private final String property2;

    public DemoObject(String property1, String property2) {
        this.property1 = property1;
        this.property2 = property2;
    }

    public String getProperty1() {
        return property1;
    }

    public String getProperty2() {
        return property2;
    }
}
```

This can be called through SLIMS REST:

```
curl -u user:pass http://url/slims/slimsrest/v1/rest/slimsgate/route/
slimsrest-sayHello
```

And will result in the following response:

```
{"property1":"value1","property2":"value2"}
```

3.7.2.2. Passing information to a plugin

URL parameters can be passed on to a plugin:

```
@Handler
public SlimsgateRestResponse test(
    @Header(SlimsGateKeeperConstants.SLIMS_WORKFLOW_INIT_PARAMETER) SlimsFlowInitParam
    slimsFlowInitParam) {

    return new SlimsgateRestResponse("application/text", "slimsgate says hi, parameters: "
        + slimsFlowInitParam.getInputParameterValues());
}
```

This can be called through SLIMS REST:

```
curl -u user:pass http://url/slims/slimsrest/v1/rest/slimsgate/route/
slimsrest-sayHello\?input1=value1\&input2=value2
```

And will result in the following response:

```
slimsgate says hi, parameters: {SLIMS_CURRENT_USER=user,
input2=value2, input1=value1}
```

3.7.2.3. Posting a body to the plugin

It is also possible to post a body to a plugin:

```
curl -u user:pass -H "Content-Type: application/json" -X POST -d
 '{"some":"body"}' http://url/slims/slimsrest/v1/rest/slimsgate/route/
slimsrest-sayHello
```

This will result in the following response:

```
slimsgate says hi, parameters: {SLIMS_CURRENT_USER=user,
body={"some":"body"}}
```

3.7.2.4. Running this asynchronously

If you do not want to wait for the SLIMS GATE route to complete and you do not need an immediate answer, you can run it asynchronously by suffixing with `/async`. You will not get a response back. This is only available when you using a POST or a PUT.

```
curl -u user:pass -H "Content-Type: application/json" -X POST -d
 '{"some":"body"}' http://url/slims/slimsrest/v1/rest/slimsgate/route/
slimsrest-sayHello/async
```

3.7.3. Starting a SLIMS GATE flow via SLIMS REST

Similar to starting SLIMS GATE routes (see above) it also possible to start SLIMS GATE flows using SLIMS REST. The advantages are:

1. You have SlimsProxy available
2. The SLIMS GATE flow executions are registered and can be tracked via the SLIMS GATE monitoring module in the SLIMS UI.

To do so, you simply need to wrap your route in a small `slimsgate.xml` file:

```
<slimsgate>
  <slimsgate-flow>
    <id>say-hello</id>
    <name>Say hello</name>
    <usage>internal</usage>
    <step>
      <name>Say hello</name>
      <hidden>true</hidden>
      <process>
        <route>slimsrest-sayHello</route>
      </process>
    </step>
  </slimsgate-flow>
</slimsgate>
```

You can then run the following:

```
~ curl -u user:pass http://url/slims/slimsrest/v1/rest/slimsgate/flow/
say-hello/slimsrest-sayHello
```

This will result in the following response:

```
slimsgate says hi, parameters: {SLIMS_CURRENT_USER=user,
flowInformation={flowStepIndex=0, flowId=say-hello,
flowRunGuid=d7d028c7-efb5-41e0-8ec0-beb61bd1bc06}}
```

3.8. Using custom fonts in Jasper

Jasper Reports generated by SLIMS can by default only use the font DejaVu. This font has no support for non Latin characters, e.g. Japanese fonts. It is possible to use custom fonts in Jasper by following these steps:

3.8.1. Get a custom font jar file

A jar file with the desired font needs to be available. The SLIMS customer zone has some font jar files available for use: <https://customer.slims.agilent.com/customer/font-jars/> . Alternatively you can build your own font jar file following these guidelines: https://community.jaspersoft.com/wiki/custom-font-font-extension#Publish_the_Font_to_the_Server . You can also contact Agilent support for assistance to build such a custom font jar file. The Work Instruction 'Creating custom font jar files for Jasper Report in SLIMS' will be used.

3.8.2. Upload the jar file to the SLIMS server

Copy the font jar file to the SLIMS server. Put it in the libs folder of your slims, slims-gate and/or requester-portal installations (create the folder if needed: C:\Program Files (x86)\Agilent Technologies\Slims\slims\libs, C:\Program Files (x86)\Agilent Technologies\Slims\slimsgate\libs and C:\Program Files (x86)\Agilent Technologies\Slims\requester-portal\libs by default on Windows or /opt/slims/libs, /opt/slimsgate/libs and /opt/requester-portal/libs on Linux) and restart SLIMS.

3.8.3. Use the font in Jasper

When editing the jrxml files, refer to the correct font like this:

```
<staticText>
  <reportElement x="248" y="26" width="100" height="30" uuid="b14d5f9d-4f37-452b-8cb9-
d01fe78948e9"/>
  <textElement>
    <font fontName="HiraginoSansIH"/>
  </textElement>
  <text>####</text>
</staticText>
```

4. SLIMS REST Development

SLIMS REST development targets engineers who wish to develop applications which interact with the SLIMS database through its REST API.

4.1. Introduction

The SLIMS REST application can be installed following the correct procedure in the SLIMS 7.2 Installation, Upgrade, Operation and Maintenance Manual. The SLIMS REST API allows basic database operations, like fetching, updating, adding and removing records. It also allows starting SLIMS GATE flows.

SLIMS REST requests are submitted to the SLIMS REST application through URLs like

```
slims.server.com/slims/slimsrest/v1/rest/user/1
```

which returns a JSON object [<http://www.json.org/>] representing the User with primary key equal to 1, for example:

```
{
  "entities": [
    {
      "links": [
        {
          "rel": "self",
          "href": "http://chl.agilent.com/slims/slimsrest/v1/rest/User/1"
        },
        {
          "rel": "user_fk_role",
          "href": "http://chl.agilent.com/slims/slimsrest/v1/rest/Role/1"
        },
        {
          "rel": "attachments",
          "href": "http://chl.agilent.com/slims/slimsrest/v1/rest/attachment/User/1"
        }
      ],
      "tableName": "User",
      "columns": [
        {
          "name": "user_userName",
          "title": "User name",
          "position": 0,
          "value": "SlimsGate",
          "hidden": false,
          "editable": true
        },
        {
          "name": "user_passwordDirty",
          "title": "Require password change on next login",
          "position": 1,
          "value": false,
          "hidden": false,
          "editable": true
        },
        {
          "name": "user_lastName",
          "title": "Last name",
          "position": 2,
          "value": null,
          "hidden": false,

```

```

"editable": true
},
.....
],
"canUpdate": true,
"canDelete": true
}
]
}

```

The returned JSON object contains a list of columns, each described by a set of properties, like "name", "title", "position", etc.; in case of foreign key fields, also a SLIMS REST request for fetching the corresponding value is included. The datatype of the returned column is mentioned.

Note that user authentication is requested when submitting REST queries.

4.2. SLIMS REST API documentation

4.2.1. Automatic documentation

The SLIMS REST API is auto documented and the documentation is available under

`slims.server.com/slims/slimsrest/v1/doc`

through an interactive interface (see Figure 4.1).

Figure 4.1. REST API documentation

Rest API For SLims		
Simple Rest API For slims to do basic database operations Contact the developer		
basic-table-operations : Basic table operations Show/Hide List Operations Expand Operations Raw		
POST	/(table)	Update multiple records
PUT	/(table)	Add a record
GET	/(table)	Fetch records by criteria
DELETE	/(table)/(pk)	Remove a record by pk
POST	/(table)/(pk)	Update a record by pk
GET	/(table)/(pks)	Fetch records by primary key
content-linked-in-eln : Content linked in ELN Show/Hide List Operations Expand Operations Raw		
GET	/eln/content/(usage)/(experimentRunStepPk)	Fetch content linked to experiment run step
DELETE	/eln/content/(usage)/(experimentRunStepPk)/(contentPk)	Unlinks content to an experiment run step
POST	/eln/content/(usage)/(experimentRunStepPk)/(contentPk)	Links content to an experiment run step
eln : Eln Show/Hide List Operations Expand Operations Raw		
GET	/eln	Fetch root of ELN Tree
GET	/eln/(table)/(pk)	Fetch children of a record in ELN Tree
POST	/eln/ExperimentRun/(pk)	Add a simplified block to an experiment run
file-repository-operations : File repository operations Show/Hide List Operations Expand Operations Raw		
POST	/repo	Upload a new attachment
GET	/repo/(pk)	Download the attachment

Within the interface it is possible to try out requests and read the returned objects.

4.2.2. Offline documentation

An offline version of the SLIMS REST API documentation is available upon request (SLIMS REST API Manual)

4.2.3. Remarks

Some SLIMS tables are restricted and can not be queried or updated. This is the list of restricted tables:

- Configuration
- Changelog
- ChangelogEntry
- ChangelogTransaction
- Dimension
- Datatype
- DbTable
- Datatype
- LabSetting
- ImportedPackage
- PackageInstallLog
- RightsSubGroup
- RightsUserGroup
- Slims
- SlimsApiClient
- SlimsPlugin
- UserGroup

The behaviour of the SLIMS REST API can be slightly modified using custom tomcat parameters. Details can be found in the SLIMS 7.2 Installation, Upgrade, Operation and Maintenance Manual, section 'Additional tomcat runtime parameters'.

4.3. Examples of using REST Requests

REST API can be embedded in scripts within html pages in order to easily make fetches on SLIMS. For example, the following html page:

```
<html>
<script src="http://code.jquery.com/jquery-latest.min.js"></script>
<script>
  $(document).ready(function() {
    var url = 'http://admin:admin@slims.customer.com/slims/slimsrest/v1/rest/user?
callback=?';
    $.ajax({
```



```
    dataType: "jsonp",
    url: url,
    jsonpCallback: "localJsonpCallback"
  });
});

function localJsonpCallback(json) {
  $.each(json.entities, function(i){
    var columns = json.entities[i].columns;
    $.each(columns, function(j) {
      var column = columns[j];
      if(column.name === 'user_userName') {
        $(''.heya').append('<p>' + column.value + '</p>');
      }
    });
  });
}
</script>
<body>
  <div class="heya"></div>
</body>
</html>
```

will displays the list of SLIMS users in a web browser.

Where the URL `slims.customer.com` needs to be substituted with the right one.

5. SLIMS Python Development

SLIMS Python development targets engineers who wish to develop applications which interact with the SLIMS database through its Python API.

5.1. SLIMS Python API

The `slims-python-api` is a project that allows users to interact easily with SLIMS using Python scripting.

To be able to install the Python API, the user has to install supported Python version 3.9 together with the corresponding pip (this is the package manager for python).

5.1.1. Python API installation

1. Install Python following the instructions from <https://realpython.com> [<https://realpython.com/installing-python/>].
2. After installing the preferred Python version, pip and the SLIMS Python API can be installed using the following commands.

```
python --version                #check if you are using the right Python version
wget https://bootstrap.pypa.io/get-pip.py    #download pip
python get-pip.py
python -m pip install -U pip          #install pip
pip3 install 'slims-python-api>=7.2.0,<7.3.0'  #install SLIMS Python API for 7.2
python3                               #Test installation for e.g. python3
>>> from slims.slims import Slims
```

5.1.2. Python API upgrade

Remove the current SLIMS Python API version

```
pip3 uninstall slims-python-api
```

Install the SLIMS Python API version

```
pip3 install 'slims-python-api>=7.2.0,<7.3.0'    #install SLIMS Python API for 7.2
```

5.1.3. Python API versions

To download a specific version of the `slims` Python api, use following examples

```
pip3 install slims-python-api                #install latest Python API version
pip3 install 'slims-python-api>=6.9.0,<7.0.0'  #install SLIMS Python API for 6.9
pip3 install 'slims-python-api>=6.8.0,<6.9.0'  #install SLIMS Python API for 6.8
pip3 install 'slims-python-api>=6.7.0,<6.8.0'  #install SLIMS Python API for 6.7
pip3 install 'slims-python-api>=6.6.0,<6.7.0'  #install SLIMS Python API for 6.6
```

5.1.4. Python API Online Documentation

The online documentation for the latest version of the SLIMS Python API can be found on `slims-python-api.readthedocs.io` [<https://slims-python-api.readthedocs.io/en/latest/>].

Here, users can find an introduction to the API, some simple examples and the full documentation.

The online cookbook for the latest version of the SLIMS Python API can be found on `slims-python-api master cookbook` [<https://github.com/genohm/slims-python-api/blob/master/cookbook>].

6. Groovy Script Development

This section contains a short introduction to the Groovy language, descriptions of methods and variables that are available to help access SLIMS data, and detailed examples of common usages.

6.1. Introduction to Groovy

Groovy is a scripting language based on Java. Most valid Java code is a valid Groovy code. Additionally, Groovy code can be made more compact and readable than Java. It is preferred to use the Fluent Groovy API over `daoHelper` when possible because it is so much easier to write and maintain. However, `daoHelper` is still available if a method is not yet available in the Fluent Groovy API.

The main things to know about Groovy that are relevant in our context are:

- **Variables:** Defining the type of the variable is not required and Groovy will assign the type automatically depending on the value. For example:

```
number = 5 // similar to: int number = 5;
text = "text" // similar to: String text = "text";
```

- **Logical statements and looping:** Groovy supports the use of logical statement as 'if,' 'else,' and looping: 'for.' Their syntax can be used in the same way as when writing Java code. Please refer to the official Groovy documentation for *Control Structures* [http://groovy-lang.org/semantics.html#_control_structures] for more details.
- **Methods:** Methods are defined by simply using the expression 'def.' For example, a method that prints in the log "Hello userName" (userName is the input parameter) is defined as following:

```
def printHello(userName) {
    log.info("Hello " + userName)
}
```

△ The message is printed in the SLIMS log file (slims.log) available on the server and it is not shown to the user in the GUI. More details on the 'log' variable will be explained in the *log section*.

A message using

```
log.info("My message")
```

can also be printed in the console directly in SLIMS. The console can be opened by clicking

the console icon:  available on the top right of the groovy script box.

Groovy scripting in SLIMS also offers some predefined methods which are available from the 'Insert' menu on the top left of the groovy scripting box. These methods will be described in section *Build-in SLIMS methods and variables*. In addition to the predefined methods, there is also a Fluent Groovy API that will be discussed later in this section.

For more details, please refer to the *Groovy official documentation* [<http://groovy-lang.org/documentation.html>].

6.2. Groovy Editor

6.2.1. Fields Where Groovy is Available

The following Role Access rights can be configured for Groovy:

Table 6.1. Permissions for Value Expressions

Access rights Category	Access rights Name	Purpose
Setup: Miscellaneous	Can Edit Value Expressions	When enabled, the role user is able to add and update Groovy expressions in value expression fields. When disabled, the role user is able to see expressions written by users with this permission, but is not able to add or update Groovy expressions in value expression fields.

Fields that require a value expression in the Groovy language are editable in an advanced source code editor. Those fields are:

- Value expression (tbfl_groovy) in Fields module.
- Dynamic filter expression (tbfl_pickListGroovy) in Fields module.
- Script (lbtm_script) in Label templates module.
- Value expression (lbtI_script) in Label template elements module.
- Conditional value expression (rule_condition) in Rules module.
- Value expression (rule_expression) in Rules module.
- Value expression (test_groovy) in Tests module.
- Value expression to convert from original unit (unit_fromOriginalUnitGroovy) in Units module.
- Value expression to convert to original unit (unit_toOriginalUnitGroovy) in Units module.

6.2.2. Editor

The header of the editor contains the following options:

- *Insert*: Opens a menu with all available/supported variables and functions.
- *Value expression console*: Opens a non-modal pop-up window that displays live Groovy logging and problems for the executions of the current value expression. Also displays the UID of the user who initiated the execution.
- *Maximize*: Maximizes the editor in order to provide a better overview of your value expression.

The editor furthermore supports:

- Syntax highlighting for the Groovy language.
- Search and Replace (Ctrl/cmd + F).
- Auto-completion (Ctrl + Space) of the following type of words:
 - *SLIMS*: cfr. variables and functions that can be added with the above 'Insert' menu.
 - *local*: already typed words in the current editor.
 - *keyword*: basic Groovy language keywords.

6.3. Introduction to Fluent Groovy API

The Fluent Groovy API was developed to make fetching SLIMS data easier by providing a more intuitive way of writing expressions. Often, the most complicated part of writing Groovy is fetching the desired entities to use in a calculation, because it requires the author to be familiar with the structure of the SLIMS database. The Fluent Groovy API benefits from pre-built methods that are more abstracted from the database schema than older daoHelper methods.

While many tables in SLIMS benefit from the API and it is continually expanding, daoHelper expressions can still be used in cases where a required Fluent Groovy API method is not yet available.

① The following sections include examples and common methods. An exhaustive list of classes and methods available in the API can be found in javadoc format here: **Complete Fluent Groovy API Manual** [../fluentDoc/index.html].

6.3.1. Advantages of Fluent Groovy API

6.3.1.1. Convenience

Complicated relationships, whether direct or indirect, are fetched in a more optimized way to improve performance. For example, contents now support the method `.allDerivations()` [../fluentDoc/com/genohm/slims/server/dao/helper/fluent/impl/links/FluentContentLinks.html#allDerivations()]. With daoHelper, you would likely loop; fetching one "generation" of records at a time. This is complex, inefficient, and requires knowledge of fields not visible in the SLIMS UI, like `cntn_fk_originalContent`:

```
def fetchAllChildren(Long contentPk) {
    def result = []
    def children = daoHelper.fetch("Content",
        SlimsRestrictions.equals("cntn_fk_originalContent", contentPk))

    if (children.isEmpty()) {
        return result
    }

    children.each { child ->
        result << child
        result.addAll(fetchAllChildren(child?.cntn_pk))
    }

    return result
}
```

```
}  
  
allChildren = fetchAllChildren(content?.cntn_pk)
```

The Fluent method `.allDerivations()` will perform a more efficient fetch which returns every downstream content in a given content's derivation tree, without you having to know the database structure used to relate derivations to each other or the programming concept of Recursion.

```
allChildren = content.retrieve().allDerivations().getAll()
```

You also cannot accidentally use non-existing fields with Fluent Groovy API methods because the Groovy will fail to execute and will throw an error.

6.3.1.2. Optimized by default

By default, queries made by `daoHelper` give you "everything" about the records you fetch. Their default field values, custom field values, display values for Dynamic Choice fields, etc. This makes your code run slower, and if the code is slow enough, it can lead to longer add and update times in your instance. You can optimize these queries by excluding data you don't actually need, but this is rarely done proactively.

In contrast, Queries made using the Fluent API are designed to be as efficient as possible by default:

1. They include only values from the records' default fields. User-created fields can still be included, but you have to explicitly request them. This prevents you from accidentally asking for more data than you need, which helps keep your instance performing smoothly.
 - To fetch data from custom fields, you can cherry-pick them by using the method `.includeFields()` [[../../fluentDoc/com/genohm/slims/server/dao/helper/fluent/FluentTableLinks.html#includeFields\(java.lang.String...\)](#)] or `.includeField()` [[../../fluentDoc/com/genohm/slims/server/dao/helper/fluent/api/FluentLimitFieldsApi.html#includeField\(java.lang.String\)](#)]
 - The option to `.includeAllFields()` [[../../fluentDoc/com/genohm/slims/server/dao/helper/fluent/FluentTableLinks.html#includeAllFields\(\)](#)] is still available if needed.
2. They do not include display values from Dynamic Choice fields. These can also be requested, using the method `includeDisplayFields()`. [[../../fluentDoc/com/genohm/slims/server/dao/helper/fluent/FluentTableLinks.html#includeDisplayFields\(boolean\)](#)] in your fetch
3. Queries made with the Fluent API cache the link between the UID and pk. This means the first execution of a given fetch will take its full execution time, but every execution afterward will occur instantly.

You can also chain fetches to cut out intermediate ones, which is supported by more-efficient joining done automatically on the backend.

6.3.1.3. Human-readable syntax

Fluent Groovy queries generally consist of: Starting record, target records, and whether you get one entity, a list of entities, or a count. What gets returned depends on the record you start with and on the operations used in your Groovy. For example:

```
listOfDerivations = content.retrieve() // Starting record
    .allDerivations() // Target records (all derivations downstream of the Content record you
    start with)
    .getAll() // Get back a list of entities
```

You can include filtering and ordering in your queries. One important note with Fluent Groovy API methods is that when you want to filter by a Dynamic Choice field's value (for example, the Content Type of your Content) you can supply the primary key of the Content Type to filter to, as is required with daoHelper, or its unique identifier (String, visible in the UI). For example:

```
numberOfBloodDerivations = content.retrieve() // Starting record
    .allDerivations() // Target records
    .withContentType("cntp_blood") // Filter applied to target records
    .count() // Get back a count of the entities meeting the criteria

content.retrieve() // Starting record
    .results() // Target records
    .withTest("test_concentration") // Filter applied to target records
    .withAnalyte("nlyt_mercury") // Filter applied to target records
    .orderDescending("rslt_valueEnteredOn") // Ordering applied to target records
    .getFirst() // Get back the result with the largest value in "rslt_valueEnteredOn" --
    i.e., the first one from our ordered list
```

Much of this was possible before with daoHelper, but the syntax for even simple, common queries is much more aligned with human speech. This makes it much easier to follow what a given script is doing when reading it for the first time. For example, fetching "test_concentration" Results linked to a given Content the old way:

```
concentrationTest = daoHelper.findByUniqueIdentifier("Test", "test_concentration")

results = daoHelper.fetch("Result", SlimsRestrictions.and(
    SlimsRestrictions.equals("rslt_fk_content", content?.cntn_pk),
    SlimsRestrictions.equals("rslt_fk_test", concentrationTest?.test_pk),
))
```

vs. with the Fluent Groovy API:

```
results = content.retrieve()
    .results()
    .includeFields("rslt_cf_myCustomField1", "rslt_cf_myCustomField2")
    .withTest("test_concentration")
    .getAll()
```

Fluent Groovy consists of: Source table, target table, and whether you get one entity or a list. For example:

```
content.retrieve()
    .allDerivations()
    .withContentType("cntp_blood")
    .getAll()
```

What gets returned depends on the contents you start with and on the operations used in your Groovy. You can include filtering, ordering, and must finish with the GET instruction.


```

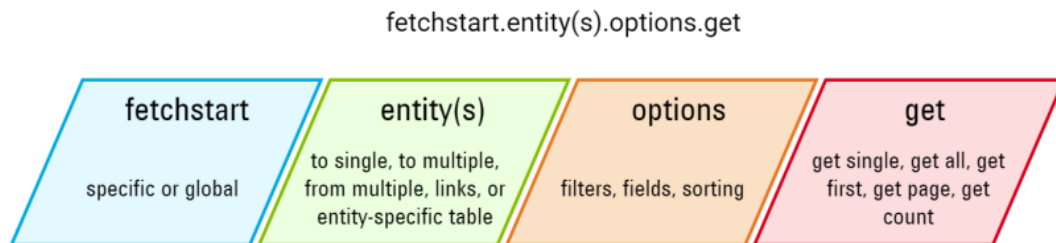
content.retrieve()
    .results()
    .withTest("test_concentration")
    .withAnalyte("nlyt_mercury")
    .orderDescending("rslt_valueEnteredOn")
    .getFirst()

```

6.3.2. How to Generally Build Fluent Groovy Expressions

The diagram below illustrates how to build a generic Fluent Groovy API expression.

Figure 6.1. Diagram with Generic Fluent Groovy API



1. The **fetch instruction** comes first and can be one of two types. `recordMap.retrieve()` uses the current record to retrieve other entities that are linked through some relations to the initial entity. Conversely, `global` is used to map more broadly in a similar way to Dao-Helper.
2. An **entity(s)** method comes second in the Fluent Groovy expression and is used to identify the single or multiple entities the expression should retrieve. Note that `global` maps to the desired table with the method `. [entityTableName] ()`. The methods listed below are used to fetch entities for `recordMap.retrieve()`. There are more specific premade methods than the ones in this list that are used to go to relations for specific tables. You can find the exhaustive list in the Complete Fluent Groovy API Manual mentioned at the start of this chapter.

The "fieldname" in the following methods needs to be the foreign key of a default or custom dynamic choice field.

Method	Description
<code>.to("fieldname")</code>	Fetches entities from a dynamic choice field with a single value on a source table going to a target table. For example, from a content to a reference data dynamic choice field.
<code>.toMultiple("fieldname")</code>	Fetches entities from a dynamic choice field allowing multiple values on a source table going to a target table. For example, from a content to its multiple result values.
<code>.from("fieldname")</code>	Fetches entities in the other direction, from a dynamic choice field with a single value on a target table going to the source table. Even though the field has a single value possible, multiple values can be fetched. For example, from multiple reference data records to their content.

You can chain links within a single fetch as long as you are following single entities as the method `.to("fieldname")`. Further guidance will be provided in the Section 6.4 section.

- The methods that may follow the entity(s) are the **options**: fields, filters, and sorting. In addition to the list of methods below, other premade queries can be applied to the target entities based on the target table to specify which kind of results you're trying to retrieve. These are documented in an exhaustive list in the Complete Fluent Groovy API Manual mentioned at the start of this chapter.

Type	Method	Description
Filter	<code>.with(slimsCriteria)</code>	Filters, fields, and sorting can be defined for the entity(s) you are fetching. This method accepts the same kind of SlimsRestriction as an argument that DaoHelper uses, which allows for complex expressions to be built.
Filter	<code>.with("fieldname" value)</code>	Shortcut method to have an <code>equal</code> operator that is the most commonly used. The value can be a pk or a unique identifier, for example.
Filter	<code>.withStatusDraft()</code> or <code>.withStatusPublished</code>	A method that works with versioned entities (for example, <code>global.experimentTemplates().withStatusDraft()</code>)
Filter	<code>.with("fieldname" [value1, value2, value3])</code>	Method to pass in a collection of values which can be a list of pks or unique identifiers, for example. This is interpreted as a "one of" filter.
Field	<code>.includeField("fieldname")</code>	Default fields are included in a fetch by default. However, if custom fields are desired, they have to be specified. This method is to include a specific custom field.
Field	<code>.includeFields("fields")</code>	Includes a comma-delimited list of custom fields.
Field	<code>.includeDisplayFields(true)</code>	<i>False by default.</i> Boolean true/false to include display fields or not.
Field	<code>.includeAllFields()</code>	Includes all custom fields (usually prevented to keep the Groovy expressions fast, but available in case all custom fields are desired).
Sort	<code>.orderAscending("fieldname")</code>	Sorting allows you to simply specify which field to sort on and which direction to sort. Can be used with <code>.getFirst()</code> to change whether you get the earliest or latest matching records.
Sort	<code>.orderDescending("fieldname")</code>	Sorting allows you to simply specify which field to sort on and which direction to sort. Can be used with <code>.getFirst()</code> to change whether you get the earliest or latest matching records.

4. Each fetch must be terminated in a **get instruction** to define how the output of the fetch is presented.

Method	Description	Returns
<code>.getSingle()</code>	Typically used for single fetches or for multiple fetches where you expect only one entity to meet the criteria(s) you set. For example, used for single entity like content which will always be single because it is unique.	A single entity that matches the criteria. If none are found, returns null. If multiple are found, returns an error.
<code>.getAll()</code>	Get all of the records that match the criteria.	List of records according to the map.
<code>.getFirst()</code>	May be used in place of <code>.getSingle()</code> such that only one entity is expected to match the criteria, but typically used instead if there is a potential that more than one entities could match the criteria. Fetches the latest entity that matches the criteria. Can be combined with the ascending or descending sort methods in order to change what it finds: latest or earliest entity that matches the criteria.	The latest entity that it finds that matches the criteria. If none are found, returns null. If multiple are found, still only returns the latest.
<code>.getPage(start, batchSize)</code>	Helpful if you expect there are a lot of records that match the criteria but want to display them all. You can build pagination to display an indexed list of records instead of all of them at one time. For example, you can view 500 records at a time out of 1500 that are returned by the Groovy.	Indexed list according to the map.
<code>.count()</code>	Finds how many records match the criteria and counts them.	Integer; how many records were found.

6.4. Guidance on Fluent Groovy API

More details on the *"log" variable* will be explained in *the log section*.

The exhaustive list of documented methods available in the API are described in the javadoc file: *Complete Fluent Groovy API Manual* [[../fluentDoc/index.html](#)].

① **Guidance on Returns:** Whether you actively include a "return" statement or not, Groovies on fields will try to return something. You can avoid situations where SLIMS returns something unexpected (ex: a `cntn_id` assigned the value of the last `daoHelper.fetch`) by ensuring there is always a return statement. If you want to return nothing, you can return the `oldValue` variable from the insert menu to keep the field at the previous value.

6.5. Build-in SLIMS Methods and Variables

SLIMS provides a certain environment with variables and methods (SLIMS API) to be used with Groovy. First, this section will show a table presenting the API provided by SLIMS and where it is available. Secondly, this section will give a detailed description with short examples on the provided API.

① **Guidance on Returns:** Whether you actively include a "return" statement or not, Groovies on fields will try to return something. You can avoid situations where SLIMS returns something unexpected (ex: a cntn_id assigned the value of the last daoHelper.fetch) by ensuring there is always a return statement. If you want to return nothing, you can return the oldValue variable from the insert menu to keep the field at the previous value.

Table 6.2. Build-in SLIMS Variables

Variables	Available In
<i>authenticatedUser</i>	<i>Field expression, Status rules, Conditional value expression rules,</i>
<i>FetchRequest</i>	<i>Field expression, Status rules, Conditional value expression rules, Result rules and Compare result rules.</i>
<i>SlimsRestrictions</i>	<i>Field expression, Status rules, Conditional value expression rules, Result rules and Compare result rules.</i>
<i>empty</i>	<i>Field expression</i>
<i>daoHelper</i>	<i>Field expression, Status rules, Conditional value expression rules, Result rules and Compare result rules.</i>
<i>error</i>	<i>Status rules, Conditional value expression rules, Result rules and Compare result rules</i>
<i>log</i>	<i>Always (Whenever Groovy can be used in SLims)</i>
<i>NA</i>	<i>Field expression</i>
<i>ok</i>	<i>Result rules and Compare result rules</i>
<i>oldValue</i>	<i>Field expression</i>
<i>sequences</i>	<i>Field expression</i>
<i>simultaneousChanges</i>	<i>Field expression</i>
<i>tableName</i>	<i>Field expression, Status rules and Conditional value expression rules</i>
<i>warn</i>	<i>Status rules, Conditional value expression rules, Result rules, Compare result rules and QC rules</i>
<i>notify</i>	<i>Status rules and Conditional value expression rules</i>
<i>email</i>	<i>Status rules and Conditional value expression rules</i>
<i>holidayHelper</i>	<i>Field expression</i>

① Tests only have the 'log' as integrated API. However, tests result values can be fetched in the value expression by using the test name followed by '.rstl_value' like the following: `testName.rstl_value`. The returned test result value will be the one obtained in the same experiment run step. If none is found then it will take the most recent one.

6.5.1. oldValue

A variable containing the value before evaluation of the field on which the field expression is calculated.

6.5.2. empty

A variable used to return an empty value in the evaluated field.

△ In fields with datatype Quantity an empty map should be returned instead: "return [:]"

△ Returning a null value in the field expressions generates an error and thus the field expression will not be evaluated. Therefore, "return empty" always has to be used instead of "return null."

6.5.3. NA

NA means that the value of the field is not available.

To set a field to NA

```
return NA
```

should be used. Note that NA is coerced to false therefore this 'if' will never be executed:

```
if (NA) { println("will never happen") }
```

To check if a field value is NA, the value of the name_of_the_field appended with `_isNa` can be checked. For example:

```
if (content.cntn_cf_field_isNa) {
    println("cntn_cf_field is NA")
}
```

will print if the value of the field is NA (and do nothing otherwise).

The expression

```
if (content.cntn_cf_field == NA)
```

can also be used *if it refers to the record the value expression is evaluated on*. Note that the expression will not work, for example, on a record fetched by a dao method (so not the same record on which the value is evaluated).

6.5.4. log

This variable is used to write messages in the SLIMS log file that is available on the SLIMS server. Please contact your system manager to get access to the SLIMS log file.

6.5.4.1. Methods Summary

Modifier and Type	Method and Description
void	error(String message)

Modifier and Type	Method and Description
	Log a message with error log level. The default log level of SLIMS is 'info' and therefore logs with this level will be written in the SLIMS log file.
	warn(String message)
void	Log a message with warn log level. The default log level of SLIMS is 'info' and therefore logs with this level will be written in the SLIMS log file.
	info(String message)
void	Log a message with info log level.
	debug(String message)
void	Log a message with debug log level. The default log level of SLIMS is 'info' and therefore this message will not be written in the SLIMS log file unless the log level of SLIMS is modified to debug.

6.5.4.2. log.error

```
void log.error(String message)
```

Log a message with error log level.

Parameters:

message - log this message.

6.5.4.3. log.warn

```
void log.warn(String message)
```

Log a message with warn log level.

Parameters:

message - log this message.

6.5.4.4. log.info

```
void log.info(String message)
```

Log a message with info log level.

Parameters:

message - log this message.

6.5.4.5. log.debug

```
void log.debug(String message)
```

Log a message with debug log level.

Parameters:

`message` - log this message.

6.5.5. tableName

Map that contains all the values of the record being evaluated and it is named after the table that the Groovy script is written for. This variable is a Map of `<String, Object>` key-value pairs where the `String` is the field name and the `Object` is the field value. Depending on the datatype of the field, the `Object` could be a `String`, `Integer`, `Quantity`, `Date`...

6.5.5.1. Methods Summary

Modifier and Type	Method and Description
	<code>get(String key)</code>
<code>Object</code>	Returns the value to which the specified key is mapped, or null if this map contains no mapping for the key.

6.5.5.2. tableName.get

```
Object tableName.get(String key)
```

Returns the value to which the specified key is mapped, or null if this map contains no mapping for the key.

① The call to this method can be done using the expression `map.key` instead of `map.get("key")`. For instance, `'content.cntn_barCode'` can be used instead of `'content.get("cntn_barCode")'`.

The list of field names used in the Groovy script is accessible through the Fields module in SLIMS.

Parameters:

`key` - the key whose associated value is to be returned

Returns:

the value to which the specified key is mapped, or `null` if this map contains no mapping for the key

Examples:

`content.cntn_id` to obtain the content id when executed on records in the *Content* table. The variable `'content'` will refer to the content for which the Groovy script is evaluated.

`result.rslt_value` to obtain the result value when executed on records in the *Result* table. The variable `'result'` will refer to the result for which the Groovy script is evaluated.

6.5.6. authenticatedUser

This variable holds a User object containing details about the authenticated user that triggered the evaluation of the field expression.

6.5.6.1. Methods Summary

Modifier and Type	Method and Description
String[]	groupNames Returns the groups' names that the user is assigned to.
String	role_name Returns the name of the role assigned to this user.
String	role_updateLevel Returns the update level of the authenticated user.
String	user_email Returns the email address of the authenticated user.
Long	user_pk Returns the primary key of the authenticated user.
String	user_userName Returns the user name of the authenticated user.

6.5.6.2. authenticatedUser.groupNames

```
String[] authenticatedUser.groupNames
```

Returns the groups' names that the user is assigned to.

Returns:

Names of the groups that the user is assigned to.

6.5.6.3. authenticatedUser.role_name

```
String authenticatedUser.role_name
```

Returns the name of the role assigned to this user.

Returns:

The name of the role assigned to this user.

6.5.6.4. authenticatedUser.role_updateLevel

```
String authenticatedUser.role_updateLevel
```


Returns the update level of the authenticated user. Possible values are: "All," "Group," "Personal," or "Nothing".

Returns:

The update level of the authenticated user.

6.5.6.5. `authenticatedUser.user_email`

```
String authenticatedUser.user_email
```

Returns the email address of the authenticated user.

Returns:

The email address of the authenticated user.

6.5.6.6. `authenticatedUser.user_pk`

```
Long authenticatedUser.user_pk
```

Returns the primary key of the authenticated user. It is the unique identifier of a user.

Returns:

The primary key of the authenticated user.

6.5.6.7. `authenticatedUser.user_userName`

```
String authenticatedUser.user_userName
```

Returns the user name of the authenticated user.

Returns:

The user name of the authenticated user.

6.5.7. `simultaneousChanges`

A variable of type `Map` that allows updating other fields of the updated record. The update is done by adding new fields' values to the `simultaneousChanges` map.

6.5.8. Quantity and Unit

6.5.8.1. `quantity(amount, unit)`

```
quantity(amount, unit)
```

Identifies the quantity, quantity field, and unit so it can be returned, used in a calculation, or created.

Groovy can be written for fields that allow choice of unit. Read about the conversions in ??? *Datatype Quantity*.

Parameters:

`amount` - The amount value to create/return.

`unit` - The unit/unit abbreviation to use for the quantity value.

Returns:

The quantity amount and unit are returned and can be used in a calculation.

Example:

A value expression on a field "Volume" (datatype Quantity, Dimension Volume) that returns "5 ml".

```
quantity("5", unit("ml"))
```

6.5.8.2. unit(String abbreviation)

```
unit(String abbreviation)
```

Used in other calculations, this simply identifies the unit abbreviation.

Parameters:

`String abbreviation` - The unit abbreviation written in parentheses, as ("x").

Returns:

Returns the unit abbreviation in the final calculation.

Examples:

A value expression on a field "Volume" (datatype Quantity, Dimension Volume) that returns complex quantity with conversion "0.005 l * 5.5 ml" aka "27.5 ml".

```
return 2 * quantity("0.005", "l") * quantity("5.5", unit("ml"))
```

A value expression on a field "Volume" (datatype Quantity, Dimension Volume) that returns calculated quantity with conversion written a different way.

```
return quantity("5", "l") .convertTo(unit("ml"))
```

6.5.9. Comparator Quantity and Position

6.5.9.1. comparatorQuantity(position, amount, unit)

```
comparatorQuantity(position, amount, unit)
```

Returns a value for the comparator quantity field with a position, amount, and unit.

Parameters:

`position` - The position (<, >, =) value to create/return. Can only be a ComparatorQuantityPosition.

`amount` - The amount value to create/return. Can be a string, number, or BigDecimal.

`unit` - The unit abbreviation to use for the quantity value.

Returns:

A quantity comparator with the specified quantity position, amount, and unit.

Example:

A value expression on a field "Mass" (datatype Comparator Quantity, Dimension Mass) that returns "< 1" in the field.

```
return comparatorQuantity(ComparatorQuantityPosition.LESS, "1", "mg")
```

6.5.9.2. ComparatorQuantityPosition

`ComparatorQuantityPosition.POSITION`

This is a helper group with the options EQUAL, LESS, or GREATER to be used in the comparatorQuantity expression.

Parameters:

`EQUAL` - One of three choices of position, adds equals sign into field: =.

`LESS` - One of three choices of position, adds less than sign into field: <.

`GREATER` - One of three choices of position, adds greater than sign into field: >.

Returns:

Returns the position in the final calculation.

Examples:

See the Section 6.5.9.1 example which includes ComparatorQuantityPosition.

6.5.10. triggerRecalculation

6.5.10.1. triggerRecalculation(entityName, recordPk) for fields

```
triggerRecalculation(entityName: String, recordPk: Long)
```

The field with the triggerRecalculation method must have the option "Reevaluate value expression after save to the database (not recommended)" enabled to use the triggerRecalculation API.

① It is preferred to use *triggerDelayedRecalculation* when possible.

Parameters:

`entityName` - Name of the entity type that should be recalculated.

`recordPk` - The primary key of the record to find.

Returns:

The `triggerRecalculation` method:

- is used to identify a record other than the one being added or updated and then trigger recalculation of that record's value expression,
- can be used in a condition so that it only triggers the identified record if the criteria are met,
- only re-saves a record once if this record would be triggered multiple times in the same transaction (e.g. by multiple records being saved at once), because the triggered record will only be triggered after all triggering records have finished their initial save,
- can trigger a maximum of 5000 entities by a single save.

Example:

Trigger the Lot value to re-evaluate if updated content value is not the same as the old value. If it is the same, keep the old value.

```
if (!oldValue.equals(newValues.cntn_cf_fk_lot)) {  
if (oldValue) {  
    triggerRecalculation("Content", oldValue);  
}  
triggerRecalculation("Content", content.cntn_cf_fk_lot);  
}
```

6.5.10.2. `triggerRecalculation(entityName, recordPk, separateTransaction)` for scheduled jobs

```
triggerRecalculation(entityName: String, recordPk: Long, separate-  
Transaction: Boolean)
```

Parameters:

`entityName` - Name of the entity type that should be recalculated.

`recordPk` - The primary key of the record to find.

`separateTransaction` - If set to true, the trigger recalculation will be in a separate transaction, meaning that if the re-trigger fails, the previous code (such as sending a notification or e-mail) is still executed.

Returns:

The `triggerRecalculation` method:

- is used to identify a record other than the one being added or updated and then trigger recalculation of that record's value expression,
- can be used in a condition so that it only triggers the identified record if the criteria are met,

- only re-saves a record once if this record would be triggered multiple times in the same transaction (e.g. by multiple records being saved at once), because the triggered record will only be triggered after all triggering records have finished their initial save,
- can trigger a maximum of 5000 entities by a single save.

Example:

The scheduled job "Nightly saving equipment" ensures that every night the values of the contents' "Next (event) date" fields are re-evaluated against the current date. If any contents throw an error on re-evaluation and therefore fail to save, an e-mail is sent to the user specified on these contents.

```
today = new Date()
contents = content.retrieve()
    .with(SlimsRestrictions.lt("cntn_cf_nextCalibrationDate", today))
    .with(SlimsRestrictions.lt("cntn_cf_nextMaintenanceDate", today))
    .with(SlimsRestrictions.lt("cntn_cf_nextQualificationDate", today))
    .getAll()
for (content in contents) {
    try {
        triggerRecalculation("Content", content.cntn_pk, true)
    } catch (com.genohm.slims.server.dao.common.ValidationException e) { // Contents that
        // cause exceptions when the scheduled job is executed, are not saved and the users must
        // be informed about this.
        parameters = ["badContentId": content.cntn_id,
                     "equipmentOwner": daoHelper.find("User",
                                                       content.cntn_cf_fk_equipmentOwner).user_email]
        email("Contents verification", parameters)
    }
}
```

6.5.11. triggerDelayedRecalculation

triggerDelayedRecalculation(entityName: String, recordPk: Long)

① triggerDelayedRecalculation can only be used in a rule of type conditional value expression.

Parameters:

entityName - Name of the entity type that should be recalculated.

recordPk - The primary key of the record to find.

Returns:

The triggerRecalculation method:

- is used to identify a record other than the one being added or updated and then trigger recalculation of that record's value expression,
- can be used in a condition so that it only triggers the identified record if the criteria are met,
- only re-saves a record once if this record would be triggered multiple times in the same transaction (e.g. by multiple records being saved at once), because the triggered record will only be triggered after all triggering records have finished their initial save,

- can trigger a maximum of 5000 entities by a single save.

The value entered by the user that triggered the delayed recalculation is retained even if the delayed recalculation fails to occur. The logging in the Groovy console in the Rules module can be viewed to explore the outcome.

Example:

Trigger delayed the Groovies on result values to re-evaluate asynchronously when the result values are validated.

```
status = daoHelper.find("Status", result.rslt_fk_status)
if (status.stts_uniqueIdentifier == "stts_rslt_validated") {
    triggerDelayedRecalculation("Content", result.rslt_fk_content)
}
```

6.5.12. triggerDelayedSlimsGateFlow

```
triggerDelayedSlimsGateFlow(flowId: String, entityName: String,
recordPk: Long, <optionalParameters: Map>)
```

Parameters:

`flowId` - Name of the SLIMS GATE flow that should be triggered.

`entityName` - Name of the entity type that the flow executes on.

`recordPk` - The primary key of the record to find.

Returns:

This triggers a SLIMS GATE flow to execute and associates the custom behavior to certain events in SLIMS. The triggered flow runs asynchronously so the user completes their transaction first, and then the delayed flow executes.

Example:

Use a custom plugin and webhooks to tell an external system when an order has been registered. The below example operates with a status rule to trigger a delayed SLIMS GATE flow called "start-app" on the Order table. There is a second conditional value expression rule to trigger another delayed SLIMS GATE flow called "printer-forwarding-app" on the Printer job table.

Example of the first status rule:

```
triggerDelayedSlimsGateFlow("start-app", "Order", order?.ordr_pk, [these: "are",
my: "parameters"]);
```

Example of the conditional value expression of the second rule:

```
printer = daoHelper.find("Printer", printerJob?.prjo_fk_printer)
return printer?.prnt_uniqueIdentifier == "prnt_slims_printer"
    && printerJob?.prjo_pk
    && printerJob?.prjo_status == "rendered"
```

And the value expression of the second rule:

```
triggerDelayedSlimsGateFlow("printer-forwarding-app", "PrinterJob",
printerJob?.prjo_pk, [printerIp: printer?.prnt_ip]);
```

6.5.13. triggerDelayedEmail

```
triggerDelayedEmail(emailTemplateName: String, <optionalParameters:
Map>)
```

Parameters:

emailTemplateName - Name of the email template that should be triggered.

Returns:

This triggers a delayed email that waits until the associated event transaction has completed. The email template is triggered asynchronously so the user completes their transaction first, and then the delayed email executes if nothing has caused it to fail.

Example:

Send an email when an order status changes to "Completed," but only after the transaction is finished saving.

```
if (status.stts_uniqueIdentifier == "stts_ordr_completed") {
    triggerDelayedEmail("Order Completed Email")
}
```

6.5.14. findByUniqueIdentifier

All Table Queries and Table RecordQueries have a method findByUniqueIdentifier. DaoHelper has a findByUniqueIdentifier method.

Parameters:

entityName - Name of the entity type.

uniqueIdentifier - uniqueIdentifier to query for.

Returns:

Returns the record associated with the queried unique identifier. In case the table does not have a unique identifier column, a TableDoesNotHaveUniqueIdentifiersException is thrown.

Example: You can use this expression to fetch contentTypePKs. (This example uses a category field called cntn_fluid_category.)

```
fluidCategoryPk =
daoHelper.findByUniqueIdentifier("ContentType", "cntn_fluid_category").cntp_pk;
```

6.5.15. macroExecutionDetails

```
macroExecutionDetails.GROUPCHOICE()
```

This is a helper group with context information on the macro step this groovy is executed in. Null if executed outside macro context.

Parameters:

`getOriginalRecord()` Only for derive and copy macro steps. Get the record being derived or copied.

`getBatchSize()` The total amount of input records when executing a macro in batch mode.

`getBatchSeqNo()` The sequence number of the execution when executing a macro in batch mode.

`getMultiplicitySize()` The expected total amount of output records for the current input record in macro steps with output records (e.g. Derive steps).

`getMultiplicitySeqNo()` The sequence number of this output record for the current input record in macro steps with output records (e.g. Derive steps).

`getMacroPk()` Primary key of the macro.

`getMacroStepPk()` Primary key of the macro step.

`getExperimentRunStepPk()` Primary key of the experiment run step (protocol run step).

Returns:

Returns the scripted information from the context of the macro step the groovy was executed on.

Example: You can use this expression to fetch the protocol run step pk from a protocol derivation step and output the pk value into a custom field on the content that was derived by the macro. (The example does not show this setup: A custom field is created first on the Content table with a String type datatype, and "Value expression is executed before creation" is enabled on the field. Then a macro with a Derive step is created and set up to be used in a Derivation protocol step.)

```
return macroExecutionDetails?.getExperimentRunStepPk();
```

6.5.16. daoHelper

This variable holds a daoHelper object. This object has methods to query records using a SLIMS criterion or the primary key of a record.

The table name is needed in the query methods to indicate in which table the query should be done. Table names are found in the Fields module in SLIMS. Additionally, there are options for the Execution Mode that choose when the expression is evaluated.

- Value expression is evaluated before creation (default value)
- Value expression is evaluated on every save

The "Reevaluate value expression after save to the database (not recommended)" is offered primarily to use the `triggerRecalculation` API later in this section.

① When using the **table name** in the script, spaces have to be removed and only capital letters will be kept as word separators. For instance the table 'Content type' will be named 'Content-Type' in the script.

6.5.16.1. Methods Summary

Modifier and Type	Method and Description
List<Map<String, Object>>	<code>fetch(String tableName, SlimsCriterion slimsCriterion)</code> Fetches all records from a <code>tableName</code> that satisfy a <code>slimsCriterion</code> .
List<Map<String, Object>>	<code>fetch(String tableName, FetchRequest fetchRequest)</code> Fetches all records from a <code>tableName</code> that satisfy a <code>fetchRequest</code> .
Map<String, Object>	<code>find(String tableName, Long pk)</code> Queries for a record in a table <code>tableName</code> with a primary key <code>pk</code> .
FetchRequest	<code>getFetchRequest()</code> Returns the fetch request of the <code>daoHelper</code> object.
Map<String, Object>	<code>findByUniqueIdentifier()</code> Searches the record in the <code>entityName</code> table by its <code>uniqueIdentifier</code> .

① Quantity values are returned differently between `fetch` and `find` methods. For `find` methods, a map (String,Object) with amount and unit entries is returned, whereas for `fetch` methods, the Quantity object is returned.

6.5.16.2. daoHelper.fetch

```
List<Map<String, Object>> daoHelper.fetch(String tableName, SlimsCriterion slimsCriterion)
```

Fetches all records from a `tableName` that satisfy a `slimsCriterion`. `slimsCriterion` are created by using `SLimsRestrictions`

Parameters:

`tableName` - name of the table to query on. For instance: Content, Order...

`slimsCriterion`.

Returns:

List of the fetched records or `null` if nothing is found.

Example:

Fetch all contents of type DNA:

```
contentType = "DNA";
contents = daoHelper.fetch( "Content", SlimsRestrictions.equals("cntp_name",
contentType));
```

6.5.16.3. daoHelper.fetch

```
List<Map<String, Object>> daoHelper.fetch(String tableName, FetchRequest request FetchRequest)
```

Fetches all records from a `tableName` that satisfy a `fetchRequest`. Please refer to the `FetchRequest` section for more details on how to use `FetchRequest`.

Parameters:

`tableName` - name of the table to query on. For instance: Content, Order...

`fetchRequest` - contains criterion for the fetch.

Returns:

List of the fetched records or `null` if nothing is found.

Example:

Fetch all contents of type DNA:

```
contentType = "DNA";
FetchRequest request =
daoHelper.getFetchRequest().setCriterion(SlimsRestrictions.equals("cntp_name",
contentType));
contents = daoHelper.fetch("Content", request);
```

6.5.16.4. daoHelper.find

```
Map<String, Object> daoHelper.find(String tableName, Long pk)
```

Queries for a record in a table `tableName` with a primary key `pk`.

Parameters:

`tableName` - name of the table to query on. For instance: Content, Order...

`pk` - The primary key of the record to find.

Returns:

Map of the found record or `null` if nothing is found.

Example:

Find the content type record of a content:

```
//content.cntn_fk_contentType returns the primary key of the content type.
contentType = daoHelper.find("ContentType", content.cntn_fk_contentType)
```

6.5.16.5. daoHelper.getFetchRequest

```
FetchRequest daoHelper.getFetchRequest()
```

Returns:

`FetchRequest` The fetch request of the `daoHelper` object.

Example:

Create a fetch request with the criterion `cntp = "DNA"`:

```
contentType = "DNA";
FetchRequest request = daoHelper.getFetchRequest();
request.setCriterion(SlimsRestrictions.equals("cntp_name", contentType));
```

6.5.17. FetchRequest

`FetchRequest` is an object containing defined fetch criterion in order to fetch SLIMS records.

6.5.17.1. Methods Summary

Modifier and Type	Method and Description
<code>FetchRequest</code>	<code>setCriterion(SLimsCriterion criterion)</code> Set a SLIMS criterion to the fetch request.
<code>FetchRequest</code>	<code>setAddSecurityCriterion(Boolean addSecurityCriterion)</code> Defines if the security criterion concerning access rights on the authenticatedUser will be applied during the fetch.

6.5.17.2. FetchRequest.setCriterion

```
FetchRequest FetchRequest.setCriterion(SLimsCriterion criterion)
```

Set a SLIMS criterion to the fetch request.

Parameters:

`criterion` - SLIMS criterion that can be added using SLIMS restrictions

Returns:

`FetchRequest` containing the set criterion.

Example:

Create a fetch request with the criterion `cntp = "DNA"`:

```
contentType = "DNA";
FetchRequest request = daoHelper.getFetchRequest();
request.setCriterion(SlimsRestrictions.equals("cntp_name", contentType));
```

6.5.17.3. FetchRequest.setAddSecurityCriterion

```
FetchRequest FetchRequest.setAddSecurityCriterion(Boolean addSecurity-
Criterion)
```

Defines if the security criterion concerning access rights on the authenticatedUser will be applied during the fetch. The default setting is `True`. It can be set to `false` if the groovy script needs to access data of all SLIMS independently of the access rights of the authenticated user that triggers the groovy script.

Parameters:

`addSecurityCriterion` - Boolean parameter that defines whether to add access right security to the fetch or not.

Returns:

`FetchRequest`

Example:

Fetch all contents of type "DNA" without applying access right security checks on the authenticated user triggering the groovy:

```
contentType = "DNA";
FetchRequest request = daoHelper.getFetchRequest();
request.setAddSecurityCriterion(false);
request.setCriterion(SlimsRestrictions.equals("cntp_name", contentType));
contents = daoHelper.fetch("Content", request);
```

6.5.18. SlimsRestrictions

`SlimsRestrictions` is used to create `SlimsCriterion` in order to fetch for records.

6.5.18.1. Methods Summary

Modifier and Type	Method and Description
	<code>equals(String fieldName, Object value)</code>
<code>SlimsCriterion</code>	Creates a criterion where the <code>fieldName</code> value is equal to the specified value.
	<code>gt/Lt/ge/le(String fieldName, Object value)</code>
<code>SlimsCriterion</code>	Creates a criterion where the <code>fieldName</code> value is greater than/lower than/greater or equal/lower or equal to the specified value.
	<code>in(String fieldName, Object[] values)</code>
<code>SlimsCriterion</code>	Creates a criterion where the <code>fieldName</code> value is one of the specified values.
	<code>like(String fieldName, String pattern)</code>
<code>SlimsCriterion</code>	Creates a criterion where the <code>fieldName</code> value matches the pattern.
	<code>not(SlimsCriterion criterion)</code>
<code>SlimsCriterion</code>	Creates a negation to the input criterion.
<code>SlimsCriterion</code>	<code>isNaSlimsRestrictions.isNa(fieldname)</code>

Modifier and Type	Method and Description
	Checks if a field is N/A.
	and()
SlimsJunction	Creates a SlimsJunction with conjunctions.
	or()
SlimsJunction	Creates a SlimsJunction with disjunctions.

6.5.18.2. SlimsRestrictions.equals

```
SlimsCriterion SlimsRestrictions.equals(String fieldName, Object value)
```

Creates a criterion where the `fieldName` value is equal to the specified value.

Parameters:

`fieldName` - name of field in SLIMS.

`value` - value that should be matched in the field with the name `fieldName`.

Returns:

`SlimsCriterion`.

Example: Creates a criterion on the field "cntn_id" to be equal to "12345678":

```
SlimsRestrictions.equals("cntn_id", "12345678")
// Corresponding sql criterion: WHERE cntn_id = '12345678';
```

6.5.18.3. SlimsRestrictions gt/Lt/ge/le

```
SlimsCriterion SlimsRestrictions.gt(String fieldName, Object value)
```

```
SlimsCriterion SlimsRestrictions.lt(String fieldName, Object value)
```

```
SlimsCriterion SlimsRestrictions.ge(String fieldName, Object value)
```

```
SlimsCriterion SlimsRestrictions.le(String fieldName, Object value)
```

Creates a criterion where the `fieldName` value is greater than/lower than/greater or equal to/lower or equal to the specified value.

Note: Unit is not taken into account when comparing Quantity objects.

Parameters:

`fieldName` - name of field in SLIMS.

`value` - value that should be matched in the field with the name `fieldName`.

Returns:

SlimsCriterion.

Example: Creates a criterion on the field "cntn_amount" to be greater than "42":

```
SlimsRestrictions.gt("cntn_amount", "42")
// Corresponding sql criterion: WHERE cntn_amount > '42';
```

6.5.18.4. SlimsRestrictions.in

SlimsCriterion SlimsRestrictions.in(String fieldName, Object[] value)

Creates a criterion where the fieldName value is one of the specified values.

Parameters:

fieldName - name of field in SLIMS.

value - value that should be matched in the field with the name fieldName.

Returns:

SlimsCriterion.

Example:

cntn_cf_name Contains one of the values "ABC", "DEF", "GHI"

```
SlimsRestrictions.in("cntn_cf_name", ["ABC", "DEF", "GHI"])
//Corresponding sql criterion: WHERE cntn_cf_name IN ('ABC', 'DEF', 'GHI');
```

6.5.18.5. SlimsRestrictions.like

SlimsCriterion SlimsRestrictions.like(String fieldName, String pattern)

Creates a criterion where the fieldName value matches the pattern.

① The percentage (%) is a wild-card character used to indicate that the pattern could contain at this position: nothing, single or multiple characters.

Parameters:

fieldName - name of field in SLIMS.

value - value that should be matched in the field with the name fieldName.

Returns:

SlimsCriterion.

Example:

cntn_cf_note Contains the pattern "%C"

```
SlimsRestrictions.like("cntn_cf_note", "%C")
```

```
//Corresponding sql criterion: WHERE cntn_cf_note LIKE '%C';
```

6.5.18.6. SlimsRestrictions.not

```
SlimsCriterion SlimsRestrictions.not(SlimsCriterion slimsCriterion)
```

Creates a negation to the input criterion.

Parameters:

fieldName - name of field in SLIMS.

value - value that should be matched in the field with the name fieldName.

Returns:

SlimsCriterion.

Example:

```
not(SlimsRestrictions.equals("cntn_createdBy", "12345678"))
//Corresponding sql command: WHERE NOT (cntn_createdBy = '12345678');
```

6.5.18.7. SlimsRestrictions.and

```
SlimsJunction SlimsRestrictions.and()
```

Creates a SlimsJunction with a conjunctions between the added criterion.

Returns:

SlimsJunction. This object is extended from SlimsCriterion.

Example:

Creates a SlimsJunction to add multiple SlimsRestrictions:

```
SlimsRestrictions.and().add(SlimsRestrictions.equals("cntn_createdBy", "12345678"))
.add(SlimsRestrictions.like("cntn_id", "%C")
//Corresponding sql criterion:
//WHERE cntn_createdBy = '12345678' AND cntn_id LIKE '%C';
```

6.5.18.8. SlimsRestrictions.or

```
SlimsJunction SlimsRestrictions.or()
```

Creates a SlimsJunction with disjunctions between the added criterion.

Returns:

SlimsJunction. This object is extended from SlimsCriterion.

Example:

Creates a disjunction between multiple SlimsRestrictions:

```
SlimsRestrictions.and().or(SlimsRestrictions.equals("cntn_createdBy", "12345678"))
```

```
.add(SlimsRestrictions.like("cntn_id", "%C")
//Corresponding sql criterion:
//WHERE cntn_createdBy = '12345678' OR cntn_id LIKE '%C';
```

6.5.19. Fetch Contents Affecting/Affected By a Content

This change relates to the Affecting/Affected By filter in the various Content module tabs. See the filter described in the ??? section.

① Fluent Groovy offers more ways of fetching related content, and it is recommended to use that API instead for this task. It is much simpler.

6.5.19.1. Fetch Content(s) Affected By a Content

```
myAffectedContents = daohelper.fetch("Content",
    SlimsRestrictions.equals("affectedBy", contentPk))
```

This groovy will return all contents affected by the content with primary key contentPk within myAffectedContents.

```
myAffectedContents = daohelper.fetch("Content",
    SlimsRestrictions.in("affectedBy", contentPks))
```

This groovy will return all contents affected by the content with primary keys contentPks within myAffectedContents.

6.5.19.2. Fetch Content(s) Affecting a Content

```
myAffectingContents = daohelper.fetch("Content",
    SlimsRestrictions.equals("affecting", contentPk))
```

This groovy will return all contents affecting the content with primary key contentPk within myAffectingContents.

```
myAffectingContents = daohelper.fetch("Content",
    SlimsRestrictions.in("affecting", contentPks))
```

This groovy will return all contents affecting the contents with primary keys contentPks within myAffectingContents.

6.5.19.3. Fetch Both Content(s) Affecting and Affected By a Content

```
myContentFamily = daohelper.fetch("Content",
    SlimsRestrictions.equals("affecting_or_affectedBy", contentPk))
```

This groovy will return all contents affecting and affected by the content with primary key contentPk within myContentFamily.

```
myContentsFamilies = daohelper.fetch("Content",
    SlimsRestrictions.in("affecting_or_affectedBy", contentPks))
```

This groovy will return all contents affecting and affected by the contents with primary keys contentPks within myContentFamily.

6.5.20. sequences

Sequences are groovy constructs that keep track of the number of times they have been executed. Each sequence requires a unique id. Sequences can be used in any groovy field in SLIMS. An example of a sequence statement:

```
return sequences.next("LocationNumber");
```

which could be used as a default value in a custom LocationNumber field, for instance.

6.5.20.1. Methods Summary

Modifier and Type	Method and Description
	next(String id)
Long	Checks to see if there is already a sequence with the specified id. If so, it will increment the number stored in that sequence and return it. If not, it will create a new (persistent) sequence, set it to 0, and return 0.

6.5.20.2. sequences.next

```
Long sequences.next(String id)
```

The next-method requires a String id. Every time it is called, it checks to see if there is already a sequence with that id. If so, it will increment the number stored in that sequence and return it. If not, it will create a new (persistent) sequence, set it to 0, and return 0.

Sequences can go up to Java's `long.MAX_VALUE`, after which they throw an exception.

Depending on the relevant settings for the encapsulating groovy field, sequences will increase every time a new record with this sequence is made using the relevant groovy script, or every time one is updated. Sequences will not update when the script is called upon opening the "create new" window, but the value the sequence will have on the next update is inserted into the script, in order to preview the result.

Inserting sequences into label templates will result in the sequence incrementing every time the label is sent to the printer, which may be several times if the printer is not connected, not switched on, or otherwise not responding.

Sequences are updated directly from the groovy script, which means that, for instance, a sequence in an if-block that is not executed, will not be increased. Similarly, a sequence in a for-loop will be incremented as many times as the code loops.

6.5.21. ok

Sets the rule evaluation of the evaluated result to 'Ok.' This method is available for rules of type *Result rules* and *Compare result rules*.

△ This method is evaluated only if the result belongs to a protocol run linked to an Order.

```
void ok(String test, String message)
```

Parameters:

test - name of the test on which the ok rule will be shown.

message - text message.

6.5.22. warn

This method name refers to two different methods that are used in different rule types:

- *Status rules, Conditional value expression rules, Result rules, Compare result rules* and *QC rules*: Shows a Warning message to the User. Transaction is not blocked.

```
void warn(String message)
```

Parameter:

message - text message.

- *Result rules* and *Compare result rules*: Sets the rule evaluation of the evaluated result to 'Warning'.

△ This method is evaluated only if the result belongs to a protocol run linked to an Order.

```
void warn(String test, String message)
```

Parameters:

test - name of the test on which the ok rule will be shown.

message - text message.

6.5.23. error

This method name refers to two different methods that are used in different rule types:

- *Status rules* and *Conditional value expression rules*: Stops/fails the record update. If possible, the message will be shown in the form into the according field.

```
void error(String field, String message)
```

Parameters:

field - the field on which the error message will be shown.

message - text message.

- *Result rules* and *Compare result rules*: Sets the rule evaluation of the evaluated result to 'Error'.

△ This method is evaluated only if the result belongs to a protocol run linked to an Order.

```
void error(String test, String message)
```

Parameters:

test - name of the test on which the ok rule will be shown.

message - text message.

6.5.24. notify

Sends a notification. This method is available for rules of type *Status rules* and *Conditional value expression rules*.

The notification does not block the current operation.

```
void notify(String title, String text, Notification.NotificationLevel level, Long[] userPks)
```

Parameters:

title - the title of the notification.

text - the text of the notification.

level (optional) - the level of the notification: INFO, SUCCESS, WARNING or ERROR. If level is not set, INFO will be used by default.

userPks (optional) - the list of users the notification should be sent to. If userPks is not set, the current user will be used by default.

Simple example:

```
notify(title: "Experiment signed", text: "Experiment has been signed off");
```

Advanced example:

```
notify(title: "Order validated", text: "Order with barcode $barcode has been validated", level: 'SUCCESS', userPks: [adminUser.user_pk, technicianUser.user_pk]);
```

6.5.25. email

Sends an email. This method is available for rules of type *Status rules* and *Conditional value expression rules*

△ In order to deliver emails, it is necessary to set the *Email templates SMTP URI* in the Lab Settings and to define an email template. See *Email Templates* for details.

```
void email(String templateName)
```

Parameter:

templateName - name of the template that builds the email(s).

6.5.26. holidayHelper

This variable can be used to calculate new dates based on working days or to calculate the date difference in working days. To correctly skip holidays, the Holiday module in SLIMS needs to be kept up-to-date.

Calculate a new date

For example, to calculate the date that is 6 working days from the current date, while excluding both weekend and holidays, the following *value expression* can be used:

```
excludeWeekends = true;
excludeHolidays true;
dateCalculator = holidayHelper.createDateCalculator(excludeWeekends,
excludeHolidays);
dateCalculator.setStartDate(org.joda.time.LocalDate.now());
dateCalculator.moveByBusinessDays(6);
newDateOnlyWorkingDays =
dateCalculator.getCurrentBusinessDate().toDateTimeAtStartOfDay().toDate();
```

To use a different date field as the start date of the calculation, replace `org.joda.time.LocalDate.now()` with `org.joda.time.LocalDate.fromDateFields(dateField)`.

Calculate working days date difference

To calculate the number of working days between two given dates, the *value expression* needs to return the following:

```
excludeWeekends = true;
excludeHolidays true;
dateDifferenceInWorkingDays =
holidayHelper.createPeriodCalculator(excludeWeekends, excludeHolidays).dayDiff(dateField1,
dateField2)
```

6.6. Examples

In the following section, use case examples of Groovy scripts applied with Field expressions and Rules will be shown.

6.6.1. Fields, datatype Quantity, table Content: calculate mass from Volume and Concentration field values

Context: Fields, datatype Quantity, table Content, calculated at every save.

```
//Verify that volume and concentration values are not null otherwise an error will be thrown
if (content.cntn_cf_volume && content.cntn_cf_concentration) {
//return the calculated the mass.
return content.cntn_cf_volume * content.cntn_cf_concentration
}
//Return an empty value if volume or concentration values are null
return empty
```

① Unit conversions are managed automatically by SLIMS as long as the fields of data type 'Quantity' have been correctly configured.

△ The value expression is evaluated only when the record is modified.

6.6.2. Fields, datatype Decimal number, table Content: calculate the difference between two quantity fields and converts it to double

Context: Fields, datatype Decimal number, table Content, calculated at every save.

```
if (content.cntn_cf_mass.getAmount() != null && content.cntn_cf_massThre.getAmount() !=
    null ) {
    difference = content.cntn_cf_mass.getAmount() - content.cntn_cf_massThre.getAmount();
    return difference.toDouble();
}
```

The conversion to double is useful when the difference value is used in highlight rules; for example, when records with mass below a certain threshold need to be highlighted.

6.6.3. Field cntn_fk_group, table Content: set default group if unique

Context: Field cntn_fk_group, table Content, calculated before creation.

```
if (authenticatedUser != null && authenticatedUser.getGroupPks() != null &&
    authenticatedUser.getGroupPks().length == 1) {
    return authenticatedUser.getGroupPks()[0];
} else {
    return null;
}
```

If the authenticatedUser belongs to only one group, that group will be assigned by default to the created content, otherwise no group will be assigned by default.

6.6.4. Fields, datatype Date, table Content: calculate expiration date from production date and duration

Context: Fields, datatype Date, table Content, calculated before creation.

```
import groovy.time.TimeCategory
if (content.cntn_cf_duration) {
    duration = content.cntn_cf_duration;
}
if (content.cntn_cf_productionDate && duration) {
    Date production = content.cntn_cf_productionDate;
    use(TimeCategory){
        int months = duration;
        return production + months.month;
    }
}
```

Note the use of TimeCategory. This package allows to add months to a date.

6.6.5. Rules, Order Status: perform checks on requestables when scheduling an order

Context: Rules, type Status, table Order, status Schedule.

```

orderPk = order.ordr_pk;
if (orderPk) {
contentPk = daoHelper.fetch("OrderContent", SlimsRestrictions.equals("rdcn_fk_order",
orderPk)).get(0).get("rdcn_fk_content");
if (contentPk) {
content = daoHelper.find("Content", contentPk);
if ("Blood".equals(content.cntp_name)) {
requests = daoHelper.fetch("Request",
SlimsRestrictions.and()
.add(SlimsRestrictions.equals("rqst_fk_order", orderPk))
.add(SlimsRestrictions.equals("rqst_fk_content", contentPk)));

requestables = requests.collect{it -> it.rqbl_name};
if (requestables.contains("hematocrit") && !requestables.contains("whiteCells")) {
error("Requestables", "When hematocrit test is requested on blood, whiteCells
test should also be requested");
}
}
}
}

```

If contents of type "Blood" are linked to an Order, it won't be possible to schedule it if "hematocrit" is requested and "whiteCells" is not requested. The following message will be shown: "When hematocrit test is requested on blood, whiteCells test should also be requested".

6.6.6. Rules, Conditional value expression: check that all results are verified when stopping a step

Context: Rules, type Conditional value expression, table Protocol run step

Conditional value expression:

```

return "DONE".equals(experimentRunStep.get("xprs_status"));

```

Value expression:

```

table = daoHelper.fetch("DbTable", SlimsRestrictions.equals("dbtb_name", "Result") );
status = daoHelper.fetch( "Status",
SlimsRestrictions.and()
.add( SlimsRestrictions.equals("stts_fk_table", table.dbtb_pk) )
.add( SlimsRestrictions.equals("stts_name", "Verified") )
);

if ( daoHelper.count( "Result",
SlimsRestrictions.equals("rslt_fk_experimentRunStep", experimentRunStep.xprs_pk)
) != daoHelper.count("Result",
SlimsRestrictions.and()
.add( SlimsRestrictions.equals("rslt_fk_experimentRunStep",
experimentRunStep.xprs_pk) )
.add( SlimsRestrictions.equals("rslt_fk_status", status.stts_pk) )
) ) {
error("", "Not all the results are in status Verified. You cannot finish the step");
}

```

```
}
```

If the number of results linked to the present experiment run step is not equal to the number of *verified results* an error is thrown and the step can not be finished.

6.6.7. Rules, Conditional value expression: requires a value for a Protocol run step field when stopping the step

Context: Rules, type Conditional value expression, table Protocol run step

Conditional value expression:

```
return "DONE".equals(experimentRunStep.get("xprs_status"));
```

Value expression:

```
if ( ! experimentRunStep.get("xprs_cf_requiredField") ) {
error("xprs_cf_requiredField", "Please enter a value for required field");
}
```

6.6.8. Rules, Conditional value expression: checks that all contents are removed when stopping a Protocol run step

Context: Rules, type Conditional value expression, table Protocol run step

Conditional value expression:

```
return "DONE".equals(experimentRunStep.get("xprs_status"));
```

Value expression:

```
exprunstepcontents = daoHelper.fetch("experimentRunStepContent",
SlimsRestrictions.equals("xrsc_fk_experimentRunStep", experimentRunStep.xprs_pk));
contentMaps = exprunstepcontents.collect { daoHelper.find("Content", it.xrsc_fk_content) };
// Different kind of checks on the content fields can be defined here
notRemovedContents = contentMaps.findAll { it.cntn_status != 50 };

if (notRemovedContents.size() > 0) {
error("Contents", "Please remove all contents before to continue");
}
```

6.6.9. Rules, Conditional value expression: use differencesMap to prevent field update based on Content status

Context: Rules, type Conditional value expression, table Content

Value expression:

```
// If content status is "available"
if (content.retrieve().to('cntn_fk_status').getSingle().stts_uniqueIdentifier
== 'stts_cntn_available') {
// and field Lot Number field (identified by its unique identifier) was updated
if (differencesMap.containsKey('cntn_cf_lotNumber')) {
```

```
        // Raise an error
        error('cntn_cf_lotNumber', 'This content field cannot be updated in status
        Available.')
    }
}
```

6.6.10. Rules, Status: use differencesMap to prevent field update based on Order Status

Context: Rules, type Status, table Order, Status Registered

Value expression:

```
// if field with ord_r_cf_fieldUniqueIdentifier unique identifier was updated.
if (differencesMap.containsKey('ord_r_cf_fieldUniqueIdentifier')) {
    // Raise an error
    error('ord_r_cf_fieldUniqueIdentifier', 'This order field cannot be updated in status
    registered.')
}
```

6.6.11. Rules, Result Status: requires a comment for some kind of results

Context: Rules, Status, table Result, status Verified

```
if(result.rslt_value == "Not OK" && result.rslt_comments == null) {
    error("rslt_comments", "A comment is required when result is Not OK");
}
```

6.6.12. Fields, datatype Dynamic choice: example of Dynamic filter expression

Context: Fields, table Protocol run, Dynamic choice with Dynamic filter expression

```
targetContentType = content.cntn_cf_fk_targetContentType;
criteria = new HashMap();
criteria.put("cntn_fk_contentType", targetContentType);
return criteria;
```

The dynamic choice list for this field will be all contents of the content type defined in the field cntn_cf_fk_targetContentType.

A dynamic filter expression can also be used to filter with multiple values:

```
criteria = [
    operator:"or",
    criteria:[
        [
            fieldName:"prvd_name",
            operator:"equals",
            value:"Agilent"
        ],
        [
            fieldName:"prvd_name",
```



```

        operator: "equals",
        value: "A Lab Inc"
    ]
}
return criteria;

```

Context: Fields, Dynamic choice with Dynamic filter expression

AuthenticatedUser can be used in a dynamic filter expression on dynamic choice fields:

```

// Retrieves groups to which the user belongs to
def groupNames = authenticatedUser.getGroupNames() as List;

// UID of products to filter on
String product1UID = "prdc_first_product_uniqueIdentifier";
String product2UID = "prdc_second_product_uniqueIdentifier";
log.info(groupNames);

// Build list of products depending on user group
def products = []
if (groupNames.contains("Group A")) {
    log.info(product1UID)
    products << product1UID
} else {
    log.info(product2UID)
    products << product2UID
}

// Return criteria
[
    fieldName: "prdc_uniqueIdentifier",
    operator: "inSet",
    value: products
]

```

6.6.13. Notify and Link to records: Generate links using GUID or UID

Context: URL links can be generated to navigate a user directly to a record using the GUID. Links can otherwise be generated using the UID before records have acquired a GUID.

Restrictions on UID functionality: Only available for tables Content, Order, Experiment Run (ELN and workflows), and Experiment Run Step (ELN and workflows).

Example: The below script used in a conditional value expression rule will notify the user whenever an order is updated and include the link to the new order using the UID.

```

{
    barcode = order?.ordr_barCode
    notify(title: "Order updated", text: "<a href='http://localhost:8080/slims/Slims.html?initialModule=order&barCode=$barcode'>Order with barcode $barcode has been updated</a>",
        level: 'SUCCESS', userPk: [authenticatedUser.getUser_pk()])
}

```

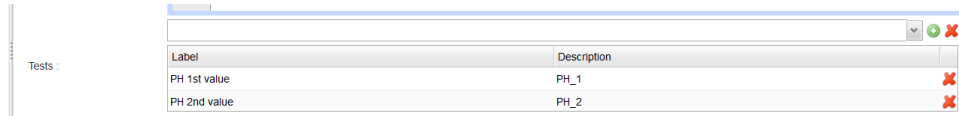
6.6.14. Test: calculate from values of other tests

Context: Test

The Value expression needs to depend on the Tests from which the value will be calculated (Figure 6.2).

```
// The expression depends on tests PH_1 and PH_2
return return (PH_1 + PH_2) / 2;
```

Figure 6.2. Test value expression dependence



Label	Description
PH 1st value	PH_1
PH 2nd value	PH_2

6.6.15. Test result: calculate result for all results of quantity test

Context: Test result

This groovy is on a result for a Test called "Concentration," which has datatype Quantity and Dimension Concentration, will return 12.3 µg/ml in the rslt_value field of all results to this test.

A more complex computation of the concentration to return (based on custom fields of the same result, on results to other tests on the same content or any other business logic) is possible using the usual Groovy tools (daoHelper, see earlier in this section).

```
// Example Groovy on Result for a Test of datatype Quantity
unit = daoHelper.fetch("Unit", SlimsRestrictions.equals("unit_abbreviation", "&mu;g/ml"))

Map<String, Object> concentrationQuantity = new hashMap<String, Object>();
concentrationQuantity.put("amount", 12.3)
concentrationQuantity.put("unit_pk", unit.get(0).unit_pk)

return concentrationQuantity;
```

6.7. Test Groovy Scripts

There is a Groovy web console available to test groovy scripts: <http://groovyconsole.appspot.com/>

All that is required is to fill in the objects/bindings the way that SLIMS would normally fill them in order to test the scripts. This way of writing scripts will easily solve typos or other compilation problems. Keep in mind that just like the other SLIMS-specific API, the Fluent Groovy API is not available when using an external Groovy console. Therefore, you cannot use this method to debug Groovy expressions and statements using this API.

Example:

A custom field "rslt_cf_expired" should be filled depending on some other result fields. This is the Groovy script used to test the script in the Groovy web console:

```
1
    result = new HashMap();
    result.put("rslt_workListDate", new Date(year: 2014, month: Calendar.APRIL, date:
1, hours: 12, minutes: 12, seconds: 0));
```

```
        result.put("rslt_cf_expiryDate", new Date(year: 2014, month: Calendar.APRIL,
date: 1, hours: 14, minutes: 12, seconds: 0));
5
        if (result?.rslt_workListDate && result?.rslt_cf_expiryDate
            &&
result.rslt_workListDate.clearTime().after(result.rslt_cf_expiryDate.clearTime())) {
            println(true);
        } else {
10             println(false);
        }
    }
```

The final Groovy script for the field in SLIMS would be:

```
1
        if (result?.rslt_workListDate && result?.rslt_cf_expiryDate
            &&
result.rslt_workListDate.clearTime().after(result.rslt_cf_expiryDate.clearTime())) {
            return true;
5        } else {
            return false;
        }
    }
```

Remarks:

- **result?.rslt_workListDate** uses the Safe Navigation Operator and will return true if result is not null and if rslt_workListDate is not null.
- **clearTime()** is a method in Groovy available for the Date class that will clear the hours, minutes and seconds.

6.8. Advanced groovy behaviour

6.8.1. Date comparisons

Extra caution should be applied when comparing dates in groovy scripts. Dates that are not yet saved in the database (for example because a user changes a date in the user interface) will be of type `java.util.Date`. Dates coming from the database (for example via `daoHelper` or the fluent API) will be of type `java.sql.Timestamp`. This means comparing them does not work

```
log.info(dateFromDatabase == dateFromUserInterface) //this will always return false
```

As a workaround you can use `getTime()`:

```
log.info(dateFromDatabase?.getTime() == dateFromUserInterface?.getTime()) //this will return true when the times are equal
```

Due to timezone conversion you might have to apply extra caution.